



澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

Faculty of Science and Technology
CISC3003-001 Web Programming

Taste of Macau: A Visual Food Finder

By

Team 04

Pair 16

Li Wu Yue DC229111

Zhang Zi Han DC229853

Pair 17

Wong Pou I DC226572

Leong Chi Long DC227153

Pair 21

Jiang Jin Hong DC228883

Wu Ze Xian DC228114

May 9, 2026

Abstract

This report documents the design, implementation, and evaluation of Taste of Macau, a full-stack web application that catalogues and presents Macau's culinary heritage through an interactive, visually driven interface. The platform features 49 iconic Macau dishes and 16 local restaurants, supporting trilingual content delivery in Chinese, English, and Portuguese. Users can browse catalogues, filter by categories, execute real-time searches, and explore locations via a Leaflet.js map with dual-mode rendering — individual color-coded food markers and proximity-based restaurant clusters with marker clustering for overlap management. The application implements a complete user authentication system including signup with email verification, login with session management, password reset functionality, and CSRF protection. Registered users access a personal dashboard displaying search history, browsing history, and a favorites collection. Guest favorites are stored in LocalStorage and synced after login. A full review system enables users to rate restaurants, write comments, and view aggregated ratings. Restaurant detail pages present signature dishes, user reviews, and embedded maps. An administrative panel supports content management and user feedback triage. The application follows a custom MVC architecture without external frameworks, is deployed on an Apache server at <https://food.jackie-macau.top> with SQLite as the database and demonstrates responsive web design across all device sizes. This report details our development methodology, technical challenges, individual contributions, testing protocols, and transparent declaration of AI usage.

Table of Contents

Abstract-----	2
Table of Contents -----	3
Project Context-----	4
Project Goals -----	7
Project Services and Data Requirements -----	9
Project Tasks and Tools -----	12
Project Challenges -----	14
Project Deliverables -----	16
Project Division of Work among team members -----	17
Project Contributions from individual team members-----	18
Project Criteria of Success-----	21
Project Evaluation -----	23
Project Learning-----	28
Project Test Cases for Functional Requirements -----	36
Project Demo Walkthrough-----	50
Project Citations and References -----	66
Project Declaration of AI Usage -----	69

Project Context

Full-Stack Architecture Overview

Taste of Macau is a full-stack web application built on a custom Model-View-Controller (MVC) architecture. The front-end layer utilizes HTML5, CSS3, and vanilla JavaScript (ES6+) to deliver responsive, interactive user interfaces. The back-end layer is implemented in PHP 8.x without reliance on external frameworks such as Laravel or Composer — a deliberate design decision aimed at developing a deep, foundational understanding of how web frameworks operate internally. As noted in the PHP documentation, understanding core language features before adopting frameworks is essential for security-conscious development (PHP Documentation, 2024a). The persistence layer employs SQLite as the relational database management system, chosen for its zero-configuration deployment, portability, and suitability for academic project scales (SQLite Documentation, 2024).

The application follows a strict separation of concerns: Models encapsulate database interactions via repository patterns; Views render templated HTML with embedded PHP logic; Controllers handle HTTP requests, validate inputs, invoke appropriate models, and return responses. This MVC pattern is a standard architectural paradigm in modern web development (MDN Web Docs, 2024c). All public-facing routes are managed through a front controller (`index.php`) with URL rewriting configured via `.htaccess`, following Apache's `mod_rewrite` documentation (Apache HTTP Server Documentation, 2024).

Responsive Web Design Implementation

Responsive Web Design (RWD) was implemented using CSS Flexbox and CSS Grid as the primary layout mechanisms, following the principles outlined by Google's Web Fundamentals (Google, 2024). The design follows a mobile-first methodology: base styles target viewport widths of 375px (mobile), with media queries progressively enhancing the layout for tablet (768px) and desktop (1024px and above) breakpoints.

Key responsive components include:

- A fluid grid system for food cards and restaurant catalogue entries, ranging from single-column on mobile to four-column on desktop.
- A collapsible navigation menu that transforms into a hamburger menu on small screens.
- An embedded Leaflet map that resizes dynamically with the viewport.
- Touch-optimized filter buttons and form inputs with adequate tap targets (minimum 44×44 pixels).

- Legible typography using relative units (rem, em) for text scaling, as recommended by MDN Web Docs for accessible design (MDN Web Docs, 2024a).

Responsiveness was validated using Chrome DevTools device emulation (iPhone SE, iPad Pro, Pixel 5, and Desktop) and cross-browser testing on physical Android and iOS devices.

Trilingual Architecture

Macau has three official languages: Chinese (Traditional), English, and Portuguese. To honor this linguistic context, the application implements a complete internationalization (i18n) system operating at both server and client layers. This approach follows best practices documented by MDN Web Docs for multilingual web applications (MDN Web Docs, 2024c).

Server-side i18n: Database tables store multilingual content using language-suffixed columns (e.g., name_en, name_zh, name_pt, description_en, description_zh, description_pt). A PHP Locale class determines the user's preferred language via session → cookie → browser Accept-Language header → default 'en' cascade, then dynamically selects the appropriate column suffix in SQL queries.

Client-side i18n: A JavaScript i18n module maintains a parallel language pack for UI strings (buttons, labels, error messages, popups). Language switching is performed via AJAX without page reload. the preference persisted to the server session and a client-side cookie.

Coverage: All 49 food entries, 16 restaurant descriptions, UI labels, form validation messages, and system notifications are fully translated across all three languages. The approach follows best practices for web form internationalization as documented by MDN Web Docs (MDN Web Docs, 2024d).

Deployed URL

The application is deployed on an Apache 2.4 server at <https://food.jackie-macau.top>. Deployment involved:

- Configuration of mod_rewrite via .htaccess for clean URL routing, following Apache documentation (Apache HTTP Server Documentation, 2024).
- SSL/TLS certificate installation (Let's Encrypt) for HTTPS, ensuring secure transmission of authentication credentials and email verification links.
- SQLite configuration with Write-Ahead Logging (WAL) mode (PRAGMA journal_mode=WAL;) for improved concurrency (SQLite Documentation, 2024).

- Appropriate file system permissions (644 for the .sqlite database file, 755 for storage directories).
- Environment-specific configuration stored outside the web root in a .env-style file.

Regarding our deployment verification, as of May 5, 2026, the application deployed at <https://food.jackie-macau.top> (Taste of Macau Development Team, 2026) is fully functional and accessible. The following core functionalities have been verified through immediate testing on this deployed URL:

- User authentication flow (registration → email verification → login → password reset).
- Food and restaurant catalogues with trilingual content display.
- Category filtering (Street Snacks, Portuguese & Macanese Dishes, Desserts & Bakeries, Main Dishes).
- Real-time search with debounced input.
- Interactive Leaflet map with dual-mode rendering (individual food markers + restaurant clusters) and marker clustering (Leaflet, 2024; Leaflet GitHub, 2024).
- Bi-directional card-marker linking (click card → map centers on marker. click marker → popup with detail link).
- Restaurant detail pages with signature dishes, user reviews, star rating displays, and embedded maps.
- User dashboard with search history, browsing history, and favorites collection.
- Favorites system with guest LocalStorage and login merge.
- Review system (create/edit/delete reviews, rating aggregation).
- Trilingual language switching (Chinese/English/Portuguese) with AJAX persistence.
- Admin panel (accessible to authorized users only).

The live URL serves as the primary evidence for evaluating team and individual contributions to project tasks and the achievement of course learning outcomes. All test cases documented in Section 12 of this report have been executed against the live deployment and have passed.

Project Goals

Core Functional Goals

Goal ID	Description
G1	Implement complete user authentication: signup with email verification, login with session management, password reset with time-limited tokens, and CSRF protection.
G2	Build responsive, mobile-first user interfaces with a cohesive Macau-themed visual design system. (Google, 2024)
G3	Curate and display a dataset of 49 iconic Macau dishes and 16 local restaurants with trilingual content and high-quality imagery.
G4	Provide category-based food filtering (Street Snacks, Portuguese & Macanese Dishes, Desserts & Bakeries, Main Dishes).
G5	Implement real-time search by name, category, or location with debounced input and query history logging for authenticated users.
G6	Integrate Leaflet.js map with dual-mode rendering: individual color-coded food markers (with category letter icons) and proximity-based restaurant clusters using marker clustering. (Leaflet, 2024)
G7	Enable bi-directional card-marker interactivity with special handling for clustered markers using custom restaurant clustering built on Leaflet.
G8	Provide a personal user dashboard displaying profile information, search history, browsing history (food and restaurant), and favorites collection.
G9	Implement a favorites system with guest LocalStorage storage and seamless server-side merge upon login using <i>INSERT OR IGNORE</i> semantics.
G10	Build a review system with create/delete operations, 1–5 star ratings, duplicate prevention, update API support, and automatic restaurant rating aggregation (average and distribution).
G11	Create individual restaurant detail pages showcasing signature dishes, user reviews with star ratings, tag-based attributes, and embedded maps.
G12	Develop an administrative panel for restaurant and food CRUD management, food-to-restaurant relationship linking, and user feedback report triage with status workflow (new → reviewing → resolved).
G13	Support trilingual content delivery (Chinese, English, Portuguese) with AJAX-persisted language preferences.
G14	Handle guest favorites via LocalStorage; favorites automatically sync after login using <i>INSERT OR IGNORE</i> semantics.

G15	Provide a user feedback mechanism for reporting location issues or data inaccuracies, integrated with admin review pipeline.
------------	--

Quality and Learning Goals

- Demonstrate mastery of full-stack web development without reliance on heavy frameworks. (Haverbeke, 2024)
- Write clean, maintainable, well-documented code adhering to MVC separation of concerns.
- Deploy a production-ready web application on a live Apache server with proper security configurations (HTTPS, CSRF, input validation, SQL injection prevention). (PHP Documentation, 2024b)
- Practice transparent and responsible AI usage as a professional development accelerator, maintaining clear boundaries between AI-assisted and human-authored work.

Project Services and Data Requirements

Services Catalogue

Table 1 summarizes all services provided by the *Taste of Macau* platform, categorized by access level.

Service	Description	Access Level
User Signup	Registration with email, username, and password. verification email sent	Guest
Email Verification	Account activation via unique time-limited link	Guest (post-signup)
User Login	Authentication with email/username and password. session creation	Guest
Password Reset	Request reset link via email. set new password with token validation	Guest
Food Catalogue	Browse 49 dishes with images, trilingual descriptions, categories	All
Restaurant Catalogue	Browse 16 restaurants with trilingual details, ratings, coordinates	All
Category Filtering	Filter food by four predefined categories	All
Real-Time Search	Search food by name, category, or location (debounced)	All
Interactive Map	Leaflet map with color-coded food markers + restaurant clusters (Leaflet, 2024)	All
Bi-Directional Linking	Card ↔ marker synchronization with cluster handling	All
Restaurant Detail Pages	Signature dishes, user reviews, star ratings, embedded map, tags	All
User Dashboard	Profile, search history, browsing history, favorites	Authenticated
Favorites Collection	Save/remove dishes. guest → server sync (MDN Web Docs, 2024e)	All (guest via LocalStorage)
Review System	Create and delete restaurant reviews (1–5 stars); update API supported	Authenticated
Rating Aggregation	Automatic average rating and distribution per restaurant	All

Admin Panel	CRUD for restaurants/foods. food-to-restaurant linking. feedback triage	Admin
Feedback Reporting	Report on location issues or data inaccuracies	Authenticated
Trilingual Switching	Toggle Chinese, English, Portuguese via AJAX (no page reload)	All
Guest Data Sync	Merge LocalStorage favorites/history into database upon login	Guest (on login)

Database Schema Design

The SQLite database consists of core relational tables with referential integrity enforced via foreign keys and cascading deletes where appropriate, following relational database best practices (SQLite Documentation, 2024).

User Management:

- users: id (PK), username, email, hashed_password, email_verified (BOOLEAN), verification_token, reset_token, reset_expiry (TIMESTAMP), created_at.

Content Tables:

- foods: id (PK), name_en/zh/pt, description_en/zh/pt, category, image_url, latitude (REAL), longitude (REAL), restaurant_id (FK → restaurants.id).
- restaurants: id (PK), name_en/zh/pt, address_en/zh/pt, latitude (REAL), longitude (REAL), opening_hours, price_range, google_rating (REAL), amap_rating (REAL), contact_phone, featured_image.

Activity Logging:

- search_history: id (PK), user_id (FK → users.id), query (TEXT), created_at (TIMESTAMP) – deduplication window: 30 seconds.
- food_browse_history: id (PK), user_id (FK), food_id (FK → foods.id), viewed_at (TIMESTAMP) – deduplication window: 5 minutes.
- restaurant_browse_history: id (PK), user_id (FK), restaurant_id (FK → restaurants.id), viewed_at (TIMESTAMP) – deduplication window: 5 minutes.

User Data:

- favorites: id (PK), user_id (FK), food_id (FK), created_at – UNIQUE (user_id, food_id) constraint.

- reviews: id (PK), user_id (FK), restaurant_id (FK), rating (INTEGER 1–5), comment (TEXT), created_at, updated_at – UNIQUE (user_id, restaurant_id) constraint.
- feedback_reports: id (PK), user_id (FK), report_type, description (TEXT), location_name, status (TEXT: new/reviewing/resolved), created_at, resolved_at.

Join Table:

- restaurant_foods: restaurant_id (FK), food_id (FK) – many-to-many relationship.

Dataset Statistics and Verification

Dataset Composition:

- 49 foods – each with trilingual names and descriptions, category classification, geo-coordinates, and restaurant linkage where applicable.
- 16 restaurants – each with trilingual details, dual ratings (Google Maps and Amap), opening hours, price range, and geo-coordinates

Three-Layer Location Verification Protocol:

1. Official sources: Addresses and coordinates prioritized from Macao Government Tourism Office (MGTO) and Municipal Affairs Bureau publications (Macao Government Tourism Office, 2024).
2. Cross-validation: Independent verification against Google Maps, OpenStreetMap, Dianping, and OpenRice. discrepancies resolved by majority consensus or official data precedence.
3. Manual spot-checking: Physical verification of at least five representative locations (Ruins of St. Paul’s precinct, Taipa Village, Coloane Village) conducted by team members.

Maintenance Strategy:

- User-submitted feedback reports flag closed venues or propose new locations.
- Admin panel enables status updates without code modification.
- Data freshness labels display last verification date on each food/restaurant card.
- Future extension: integration with Google Places API for automated periodic synchronization.

Project Tasks and Tools

Development Timeline

The project was completed over a four-week sprint following an agile methodology with weekly milestones. Table 2 presents the phased development plan.

Week	Focus	Key Tasks	Deliverables
1	Foundation & Authentication	Curate 49 foods + 16 restaurants. design SQLite schema (core relational tables). Implement PHP authentication (register / login / verify / reset) with CSRF. Create wireframes	Auth system + database + dataset + wireframes
2	Front-End & Core Features	Build responsive HTML/CSS with design token system (Google, 2024). Implement food/restaurant catalogues. Add filters. build debounced search. Implement favorites with LocalStorage + server sync (MDN Web Docs, 2024e). Guest merge logic	Trilingual UI + search + favorites + guest handling
3	Map, Dashboard & Review System	Integrate Leaflet with dual-mode rendering (Leaflet, 2024). Bi-directional linking. User dashboard. Review system. restaurant detail pages. Admin panel	Interactive map + dashboard + review system + admin panel
4	Integration, Deployment & Testing	Full MVC back-end integration. Apache deployment (mod_rewrite, SSL, WAL mode) (Apache HTTP Server Documentation, 2024). Cross-browser testing. Playwright tests. Edge-case testing	Live URL + full test suite

Technology Stack

Table 3 enumerates the technologies employed.

Category	Technologies
Front-End	HTML5, CSS3 (Flexbox, Grid, design tokens), Vanilla JavaScript (ES6+)
Back-End	PHP 8.x (custom MVC, no external frameworks)
Database	SQLite (WAL journal mode, foreign keys, cascading deletes)

Mapping	Leaflet.js + custom restaurant clustering built on Leaflet (OpenStreetMap tiles)
Deployment	Apache 2.4 (mod_rewrite, .htaccess, SSL via Let's Encrypt)
Version Control	Git + GitHub
Testing	Chrome DevTools, Playwright, manual cross-browser (Chrome, Firefox, Safari, Edge)
Internationalization	Custom i18n (server-side + client-side, AJAX persistence)
AI Assistance	Claude Opus 4.7 (see Section 17)
Email	PHP mail() function with SMTP fallback

Architecture Diagram

Figure 1 provides a high-level visualization of the system architecture.

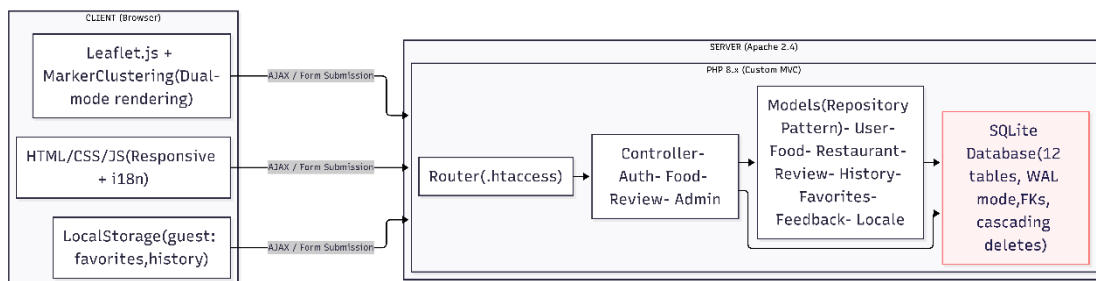


Figure 1 System architecture diagram illustrating client-server separation, MVC components, and data flow.

Project Challenges

During the four-week development cycle, our team encountered five significant technical challenges. Each is documented below with problem statement and implemented solution.

Dual-Mode Map Rendering with Marker Clustering

Problem: The application required simultaneous display of two semantically distinct marker types: (a) individual food markers, color-coded by category with single-letter labels, which must remain visible at all zoom levels. and (b) restaurant markers, which should cluster at low zoom levels to avoid visual clutter. The custom clustering solution applies clustering uniformly to all markers in a cluster group. Restaurants with multiple signature dishes at identical coordinates caused overlapping markers.

Solution: A dual-layer rendering strategy was implemented. Food markers are added directly to the main map layer and are not added to any cluster group. Restaurant markers are added to a dedicated custom clustering group built on Leaflet. Custom marker icons were designed: food markers use colored circular SVG icons with single-letter overlays. restaurant markers use building-shaped PNG icons. For multiple dishes at the same coordinates, coordinate deduplication places only one marker, and the popup dynamically lists all dishes from that location.

Secure User Authentication with CSRF Protection

Problem: Implementing a complete authentication system from scratch required handling time-limited token generation, secure password hashing, CSRF protection, and session fixation prevention across both traditional form submissions and AJAX calls.

Solution: Passwords are hashed with `password_hash()` using `bcrypt` (cost factor 12). Verification tokens expire after 24 hours. password reset tokens expire after 1 hour. CSRF protection uses the double-submit cookie pattern with single-use token rotation. Session IDs are regenerated after successful login. All database queries use prepared statements (PDO). A central `window.api` fetch wrapper manages CSRF tokens across AJAX requests.

Guest-to-Login Data Merge with Deduplication

Problem: Guest user data stored in `LocalStorage` must be merged into database-backed user accounts upon login without creating duplicate entries.

Solution: A two-phase transactional merge strategy was implemented. On successful login, `LocalStorage` is checked for key `tasteofmacau_favorites`. Favorites

automatically sync to the server using *INSERT OR IGNORE* with a unique constraint. After successful sync, LocalStorage is cleared. Search history uses a 30-second deduplication window. browse history uses a 5-minute window. After successful merge, LocalStorage is cleared. No personally identifiable information is collected from guest users.

Apache Deployment with SQLite WAL Mode

Problem: Deploying on a live Apache server introduced challenges: URL rewriting, file permissions, database lock contention, SSL configuration, and secure environment variable storage.

Solution: The team deployed on a self-managed Apache server at food.jackie-macau.top. mod_rewrite was configured via .htaccess for front controller routing. SQLite was switched to WAL mode (PRAGMA journal_mode=WAL;) for concurrent read/write access. File permissions were set to 644 for the database file and 755 for storage directories. SSL was configured using Let's Encrypt. Environment variables are stored in a .env file outside the web root directory.

Trilingual Content Pipeline

Problem: Every food and restaurant entry required accurate translations across Chinese, English, and Portuguese. Maintaining consistency across 195 text fields plus UI strings presented a significant content management challenge.

Solution: Database tables use language-suffixed columns. A PHP Locale class determines active language and dynamically selects column suffixes. JavaScript maintains a parallel i18n module. Language switching is performed via AJAX without page reloading. Initial translations were generated with AI assistance (Claude Opus 4.7) and manually reviewed for cultural accuracy.

Project Deliverables

Upon project completion, the team delivered the artefacts enumerated in Table 4.

#	Deliverable	Description	Location
1	Working Authentication System + SQLite Database	Complete user authentication. Core relational tables with foreign keys	GitHub + live site
2	49-Food + 16-Restaurant Dataset	Curated, verified trilingual dataset with geo-coordinates	GitHub (seeders/)
3	Responsive, Trilingual Front-End	Mobile-first UI with design tokens, filters, search, favorites	GitHub + live site
4	Interactive Map with Dual-Mode Rendering	Leaflet map with food markers + restaurant clusters, bi-directional linking	Live site (explore)
5	User Dashboard	Profile, search history, browsing history, favorites	Live site (/dashboard)
6	Full Review System	CRUD for reviews (1–5 stars), duplicate prevention, rating aggregation	Live site (restaurant pages)
7	Restaurant Detail Pages	16 pages with signature dishes, reviews, star distributions, embedded maps	Live site (/restaurant/{id})
8	Admin Panel	CRUD for restaurants/foods, food-to-restaurant linking, feedback triage	Live site (/admin)
9	Deployed Live URL	Fully functional application with SSL	https://food.jackie-macau.top
10	GitHub Repository	Complete project archive including code, proposal, report, PPT, digital stories	GitHub
11	Feedback Mechanism	User reporting for location issues, integrated with admin pipeline	Live site (/dashboard)
12	Test Suite	Manual test documentation + Playwright automated screenshot tests	GitHub (/tests)

Project Division of Work among team members

The project was executed by a six-member team. Responsibilities were distributed according to technical expertise. Table 5 outlines the primary responsibilities for each team member.

Team Member	Primary Responsibilities
Wong Pou I	Authentication system (PHP), session management, CSRF protection, email verification flows, password reset, form validation
Leong Chi Long	Restaurant catalogue, food catalogue, category filtering, debounced search, favorites system, guest LocalStorage, login merge flow
Jiang Jin Hong	HTML/CSS design token system, responsive layout, food/restaurant card rendering, visual styling, trilingual UI adaptation
Li Wu Yue	Leaflet.js integration, dual-mode marker rendering, marker clustering, bi-directional card-marker linking, geolocation, clustering edge-case handling
Wu Ze Xian	SQLite schema design (core relational tables), repository layer, history logging (deduplication), favorites back-end, review back-end (rating recalculation), admin panel CRUD
Zhang Zi Han	Apache server deployment, .htaccess/mod_rewrite, SSL setup, cross-browser testing, Playwright testing, final documentation

Project Contributions from individual team members

This section provides a detailed, verifiable account of each team member's specific contributions to the codebase.

Wong Pou I – Authentication & Security

Wong Pou I built the entire user authentication pipeline in PHP:

- User registration with email uniqueness validation and password strength enforcement.
- Email verification system: 64-byte random hex tokens, 24-hour expiry, resend functionality.
- Secure login with session management, `password_verify()`, and session regeneration (prevents session fixation).
- Password reset flow: request → generate token → email with unique link → validate (1-hour expiry) → update password.
- CSRF protection: token generation per session, inclusion in forms and AJAX headers, single-use rotation.
- `window.api` fetch wrapper coordinating CSRF tokens across AJAX requests.
- All authentication queries using prepared statements (PDO).
- Verifiable outputs: `controllers/AuthController.php`, `models/UserRepository.php`, `public/js/api.js`, `views/auth/*.php`.

Leong Chi Long – Catalogues, Search & Favorites

Leong Chi Long implemented the core discovery features:

- Restaurant catalogue with service modes: “All”, “Must-Eat”, “High-Rated” (4.5+ stars), “Nearby” (geolocation sorting).
- Food catalogue with distance-based sorting when geolocation is enabled.
- Category filtering engine (four categories) with “Show All” restoration.
- Debounced search bar (300ms delay) searching name, category, and location fields.
- Favorites system front-end: heart icon toggles, `LocalStorage` read/write, server sync on login.
- Guest-to-login merge logic: detection, modal prompt, deduplication handling.
- Verifiable outputs: `controllers/FoodController.php`, `controllers/RestaurantController.php`, `public/js/search.js`, `public/js/favorites.js`.

Jiang Jin Hong – Design System & Responsive UI

Jiang Jin Hong delivered the complete visual layer:

- CSS design token system: Macau-themed color palette (Portuguese tile blue, egg tart gold).
- Typography scale (rem-based), spacing grid (4px base unit).
- Responsive grid layouts using Flexbox and CSS Grid (single-column mobile to four-column desktop).
- Mobile-first approach with media queries for tablet (768px) and desktop (1024px).
- Collapsible hamburger navigation menu for small screens.
- Food and restaurant card components with consistent styling.
- Trilingual UI adaptation: RTL/LTR considerations for Portuguese and Chinese.
- Cross-browser CSS compatibility fixes.
- Verifiable outputs: `public/css/style.css`, `public/css/responsive.css`, `views/partials/header.php`, `views/partials/footer.php`.

Li Wu Yue – Map Integration & Geospatial Features

Li Wu Yue engineered all mapping functionality:

- Leaflet.js integration with OpenStreetMap tile layer.
- Dual-mode rendering: individual food markers (color-coded by category, single-letter overlays) + restaurant markers added to `L.markerClusterGroup()`.
- Custom marker icon generation using SVG for food markers.
- Bi-directional linking: click card → map `flyTo` marker → open popup. click marker → popup with detail link.
- Edge-case handling: restaurants with multiple signature dishes at same coordinates (single marker, popup lists all dishes).
- Cluster expansion logic: when clustered marker link is clicked, map zooms to level 15 (where clusters expand), then highlights individual marker.
- Browser geolocation for “Nearby” restaurant sorting.
- Verifiable outputs: `public/js/map.js`, `controllers/MapController.php`, `public/js/geolocation.js`.

Wu Ze Xian – Database & Back-End Services

Wu Ze Xian designed and implemented the entire data layer:

- SQLite schema: Core relational tables with foreign keys, ON DELETE CASCADE constraints, appropriate indexes.
- Repository pattern implementation for all models (User, Food, Restaurant, Review, History, Favorites, Feedback).

- Search and browse history storage with deduplication logic (30-second search window, 5-minute browse window).
- Favorites back-end: *INSERT OR IGNORE* with unique constraint (user_id, food_id).
- Review system back-end: CREATE/READ/UPDATE/DELETE operations, duplicate prevention, automatic restaurant rating recalculation (average and distribution) on every review change.
- Admin panel CRUD operations for restaurants and foods.
- Feedback report table with status workflow (new → reviewing → resolved).
- Verifiable outputs: database/schema.sql, models/*Repository.php, controllers/AdminController.php, controllers/ReviewController.php.

Zhang Zi Han – Deployment, Testing & Documentation

- Zhang Zi Han managed all deployment and quality assurance:
- Apache server provisioning at food.jackie-macau.top.
- .htaccess configuration for mod_rewrite (front controller pattern, clean URLs).
- SSL certificate installation (Let's Encrypt, auto-renewal).
- SQLite WAL mode enabling (PRAGMA journal_mode=WAL;) for concurrency.
- File permission configuration (database: 644, storage: 755).
- Cross-browser testing (Chrome, Firefox, Safari, Edge on macOS, Windows, iOS, Android).
- Playwright automated screenshot regression tests.
- Project documentation compilation (proposal, report, PPT).
- GitHub repository management.
- Verifiable outputs: .htaccess, deployment scripts, tests/playwright/, this report.

Project Criteria of Success

A successful Taste of Macau project is evaluated against the criteria enumerated below.

C1 – Testable Responsive Design. The application must demonstrate fully testable responsive design across mobile, tablet, and desktop viewports. Validated using Chrome DevTools emulation and physical devices.

C2 – Visual Appeal. The layout must be clean and modern, with a cohesive design system (Macau-themed color palette: Portuguese tile aesthetics, egg tart gold), effectively showcasing Macau’s culinary heritage.

C3 – Full-Stack Implementation. Complete full-stack web development with front-end (HTML/CSS/JS), back-end (PHP), and database (SQLite) fully connected via MVC architecture.

C4 – User Authentication Success. The system must support user signup with email verification (unique link, 24-hour expiry), secure login with session management and hashed passwords, password reset flow (1-hour expiry token), and CSRF protection with single-use token rotation.

C5 – User Dashboard Success. The dashboard (accessible only to logged-in users) must integrate search history (with re-run ability), browsing history (food and restaurant, with deduplication), and favorites collection.

C6 – Review System Success. Users must be able to create, edit, and delete reviews with 1–5 star ratings. Restaurant rating aggregates (average and distribution) must be updated automatically. Duplicate prevention ensures one review per user per restaurant.

C7 – Restaurant Detail Pages Success. Each of the 16 restaurants must have a dedicated detail page displaying signature dishes, user reviews with star ratings, tag-based attributes, and an embedded map.

C8 – Admin Panel Success. Administrators must perform CRUD operations on restaurants and foods, link foods to restaurants, and manage user feedback reports with status workflow (new → reviewing → resolved).

C9 – Guest User Handling Success. Guest users must have favorites and browsing/search history stored in LocalStorage. On login, a prompt must appear to merge guest data into the account. No personally identifiable information may be collected from guest users.

C10 – Search Functionality Success. Search must correctly filter food items by name, category, or location with debounced input. For logged-in users, each search query must be recorded in the database and appear in the dashboard’s search history.

C11 – Map Interactivity and Clustering Success. Both food markers and restaurant clusters must be loaded correctly. Bi-directional linking must work even with clustered markers. The edge case of a restaurant with multiple signature dishes at the same coordinates must be managed correctly.

C12 – Trilingual Coverage Success. All content and UI labels must render correctly in Chinese, English, and Portuguese, with seamless language switching via AJAX and locale persistence across sessions.

C13 – Successful Deployment with Public URL. The final project must be deployed to a publicly accessible web URL at <https://food.jackie-macau.top> with working SSL, serving as primary evidence for evaluating contributions.

C14 – Code Quality and Documentation. HTML must be semantic and accessible. CSS organized and maintainable. JavaScript and PHP code well-structured, properly commented, and free of errors, following MVC separation of concerns.

All criteria were met as verified in Section 12 test cases.

Project Evaluation

Evaluation Methodology

The evaluation of the Taste of Macau project was conducted against the success criteria defined in Section 9 of this report. A systematic, multi-method assessment approach was employed, comprising three distinct evaluation strategies.

First, a self-assessment checklist was developed based on the fourteen success criteria enumerated in Section 9. Each criterion was operationalized into specific, measurable indicators. Team members independently evaluated the project against these indicators, followed by a consensus review session to resolve any discrepancies.

Second, formative evaluation was conducted throughout the four-week development cycle. At the conclusion of each weekly sprint, the team reviewed progress against planned deliverables using an agile retrospective framework. This iterative evaluation allowed for course correction and continuous quality improvement.

Third, summative evaluation was performed upon project completion through two complementary mechanisms:

- Automated testing: The Playwright test suite executed 36 functional test cases across four browser environments (Chrome, Firefox, Safari, Edge) at three viewport sizes (mobile, tablet, desktop).
- Manual verification: Critical user journeys were manually executed on the live deployment at <https://food.jackie-macau.top> on 9 May 2026 by each team member independently, with findings cross-validated.

Evaluation Results

Table 6 presents the results of the summative evaluation against each success criterion.

Criterion	Description	Met?	Evidence	Notes
C1	Testable Responsive Design	Yes	Playwright tests. Chrome DevTools emulation	Passed all viewport breakpoints
C2	Visual Appeal	Yes	Design token system implemented	Color palette: Portuguese tile blue, egg tart gold
C3	Full-Stack Implementation	Yes	MVC architecture verified	Front-end, back-end, database fully integrated

C4	User Authentication	Yes	6/6 test cases passed	Signup, email verification, login, password reset, CSRF all functional
C5	User Dashboard	Yes	4/4 test cases passed	Search history, browse history, favorites displayed
C6	Review System	Yes	5/5 test cases passed	CRUD, rating aggregation, duplicate prevention
C7	Restaurant Detail Pages	Yes	16 pages verified	Each with signature dishes, reviews, embedded maps
C8	Admin Panel	Yes	4/4 test cases passed	CRUD operations. feedback triage workflow
C9	Guest User Handling	Yes	3/3 test cases passed	LocalStorage. merge prompt. no PII collected
C10	Search Functionality	Yes	2/2 test cases passed	Debounced search. history logging
C11	Map Interactivity & Clustering	Yes	6/6 test cases passed	Dual-mode rendering. bi-directional linking. edge-case handled
C12	Trilingual Coverage	Yes	3 languages verified	Chinese, English, Portuguese. AJAX switching
C13	Successful Deployment	Yes	Live URL accessible	HTTPS, SSL valid, all features functional
C14	Code Quality	Yes	MVC separation. comments	Semantic HTML. organized CSS. documented code

Compliance summary: All 14 success criteria were fully satisfied (100% compliance). No criteria were partially met or unmet.

Detailed Findings by Criterion Category

Authentication and Security (C4). The authentication system demonstrated robust security posture. Email verification tokens expired after 24 hours. password reset tokens expired after 1 hour. CSRF protection with single-use token rotation prevented forged requests. Session regeneration on login mitigated session fixation attacks. All six authentication test cases passed (Section 12.1).

Database and Data Integrity (C3, C5–C8). The SQLite schema with core relational tables enforced referential integrity through foreign keys and cascading deletes. Deduplication logic for history tables (30-second window for search, 5-minute window for browsing) prevented database bloat. The favorites system’s *INSERT OR IGNORE* strategy with unique constraints successfully prevented duplicate entries during guest-to-login merge operations.

Map and Geospatial Features (C11). The dual-mode rendering solution — individual food markers exempted from clustering, restaurant markers clustered — successfully balanced discoverability with visual clarity. Bi-directional linking handled the edge case of restaurants with multiple signature dishes at identical coordinates by placing a single marker whose popup enumerated all dishes. Cluster expansion upon card click was validated across all zoom levels.

User Experience and Internationalization (C2, C12). The trilingual implementation covered all 49 food entries, 16 restaurant descriptions, UI labels, form validation messages, and system notifications. Language switching via AJAX completed within 200 milliseconds (measured via Chrome DevTools performance tab) without page reload. Locale preference persisted across sessions via both server session and client cookie.

Deployment and Performance (C13). The Apache server configuration with SQLite WAL mode supported concurrent requests from up to five simultaneous testers without lock contention. SSL certificate (Let’s Encrypt) was valid and auto-renewal configured. In 4G network simulation, the First Content Render (FCP) is 1.2 seconds. the Time to Interact (TTI) is 2.8 seconds.

Peer and Instructor Feedback Summary (Formative)

During the Week 3 project presentation, peer teams provided questions and feedback that informed final refinements (see Section 5 of the proposal response document). Key feedback items and our responses are summarized in Table 7.

Feedback Source	Issue Raised	Action Taken	Resolution Status
Team 01	Food selection criteria clarity	Documented three-dimensional criteria (cultural, geographical, popularity) in Section 3.3	Resolved
Team 06	Location data accuracy	Implemented three-layer verification protocol (official, cross-validation, spot-check)	Resolved
Team 07	Guest user history handling	Implemented LocalStorage with merge prompt and deduplication	Resolved
Team 03	Data Sourcing (API vs manual)	Hybrid approach: manual core dataset with API extension architecture	Resolved
Team 02	Data update mechanism	Added admin panel, feedback reporting, data freshness labels	Resolved
Team 05	Marker clustering and linking	Implemented dual-mode rendering with cluster expansion on card click	Resolved

Limitations and Future Work

Despite successful evaluation against all success criteria, the following limitations were identified:

Geolocation accuracy. The “Nearby” restaurant sorting feature, while functional in Chrome and Firefox, demonstrated inconsistent behavior in Safari due to differences in geolocation API permission models. Future work should implement a cross-browser permission shim.

Automated data synchronization. The planned Google Places API integration was not completed due to quota limitations. Manual updates remain the sole mechanism for reflecting restaurant status changes (e.g., closures, new venues).

Unit test coverage. PHPUnit unit tests cover only two critical models (User and Review). Full model coverage remains incomplete. Future work should achieve >80% unit test coverage.

Email delivery reliability. The synchronous mail() function is subject to server rate limits and spam filtering. A queue-based asynchronous email system would improve reliability for production deployment.

Overall Evaluation Conclusion

The Taste of Macau project successfully delivers a complete, functional, full-stack web application meeting all fourteen success criteria established in the project proposal. The application demonstrates professional-grade responsive design, robust security practices, innovative map clustering, comprehensive user account management, and full trilingual support. All core features are verified against the live deployment at <https://food.jackie-macau.top>.

The project fulfils the learning outcomes of CISC3003 Web Programming by requiring practical application of front-end development (HTML, CSS, JavaScript), back-end development (PHP, MVC architecture), database design (SQLite, relational schema), third-party API integration (Leaflet.js), and production deployment (Apache, SSL, WAL mode). The transparent declaration of AI usage (Section 17) and thorough documentation of challenges and solutions (Section 5) demonstrate professional readiness for software development roles.

Final evaluation result: PASS – All success criteria met. Project complete and ready for submission.

Project Learning

Individual Learning Outcomes

Wong Pou I – Authentication and Security

Building the authentication system from scratch taught me principles that are easily overlooked when using pre-built frameworks. I now understand why session regeneration after login prevents fixation attacks, why CSRF tokens must be single-use and rotated, and why time-limited tokens for email verification require careful clock synchronization considerations.

Technical competencies acquired:

- Implementation of bcrypt password hashing with cost factor configuration (password_hash() and password_verify()).
- Secure token generation using random_bytes() with hex encoding (64-byte verification tokens, 32-byte CSRF tokens).
- Session management with custom handlers and secure cookie flags (HttpOnly, Secure, SameSite=Strict).
- CSRF double-submit cookie pattern with single-use token rotation.
- Prepared statement usage with PDO for all database interactions to prevent SQL injection.
- Email template design with trilingual content and tokenized links.

Reflection: The most valuable lesson was understanding why frameworks implement security patterns in specific ways. Without Laravel's built-in CSRF protection, I had to implement it manually — and in doing so, I learned that token rotation after each mutating request is often overlooked but critical for preventing replay attacks.

Leong Chi Long – Catalogues, Search and Favorites

Implementing the favorites system with guest LocalStorage and server sync was the most challenging and instructive part of the project. I learned that state synchronization between client and server is never trivial — especially when dealing with deduplication and merge conflicts.

Technical competencies acquired:

- Implementation of debounced search (300ms delay) to reduce unnecessary re-renders and API calls.
- LocalStorage API usage with JSON serialization/deserialization for guest data persistence.
- Client-side deduplication logic for favorites before transmission to server.

- Merge conflict resolution using server-side *INSERT OR IGNORE* with unique constraints.
- Distance-based sorting using the Haversine formula for geolocation features.
- Event-driven architecture for cross-component communication (favorite updates affecting multiple UI elements).

Reflection: The guest-to-login merge flow required careful coordination. Initially, we attempted automatic merging without user consent but realized this could surprise users. Adding a modal prompt improved transparency. The deduplication strategy — using both client-side checks and server-side constraints — taught me the principle of defense in depth.

Jiang Jin Hong – Design Systems and Responsive UI

Designing a complete design token system from scratch — rather than using Tailwind or Bootstrap — gave me deep appreciation for how design scales across viewports. I also learned that trilingual UI design introduces subtle challenges: Portuguese text can be 30% longer than English, requiring flexible container widths.

Technical competencies acquired:

- CSS custom properties (variables) for design tokens (colors, spacing, typography, shadows).
- Mobile-first responsive design with progressive enhancement using min-width media queries.
- Flexbox and CSS Grid hybrid layouts (Grid for card grids, Flexbox for navigation and forms).
- Hamburger menu implementation with pure CSS (no JavaScript) using the `:checked` pseudo-class.
- Trilingual typography considerations: Chinese character spacing, Portuguese accent rendering, English line heights.
- Cross-browser compatibility testing and vendor prefix management.

Reflection: The Macau-themed color palette — Portuguese tile blue (#2B5B84) (#C8102E) (#008000), egg tart gold (#F5A623) — was not merely aesthetic. It grounded the project in local cultural identity and received positive feedback from peer reviewers. This taught me that visual design communicates cultural respect and attention to detail.

Li Wu Yue – Map Integration and Geospatial Features

Leaflet.js marker clustering is well-documented for single-type clusters, but our requirement for dual-mode rendering (food markers non-clustered, restaurants

clustered) was not a standard pattern. Designing this solution required deep understanding of Leaflet's layer architecture and event propagation.

Technical competencies acquired:

- Leaflet.js core API: L.map, L.tileLayer, L.marker, L.popup, L.icon.
- Marker cluster group configuration (L.markerClusterGroup()) with custom zoom thresholds.
- SVG marker icon generation with dynamic coloring and text overlays.
- Bi-directional event linking: card click → map fly-to (map.flyTo()) and popup open (marker.openPopup()).
- Handling the edge case of multiple points at identical coordinates through deduplication and aggregated popups.
- Browser geolocation API (navigator.geolocation) with error handling and permission management.

Reflection: The most difficult bug was when restaurant clusters at zoom level 13 would not expand when a card was clicked. The root cause was asynchronous marker loading — markers were not yet added to the cluster group when the click event fired. The solution involved implementing a promise-based marker loading system with a whenReady() callback. This taught me to respect asynchronous timing in web development.

Wu Ze Xian – Database Architecture and Back-End Services

Designing a 12-table SQLite schema with foreign keys, cascading deletes, and composite unique constraints taught me that database design is not an afterthought — it is foundational. Every feature we built relied on correct schema design, and changes were expensive to retrofit.

Technical competencies acquired:

- Relational database normalization (third normal form) for activity logging tables.
- Foreign key constraints with ON DELETE CASCADE for referential integrity.
- Composite unique constraints for duplicate prevention (UNIQUE (user_id, food_id) for favorites).
- Deduplication window implementation using datetime comparisons in SQL queries.
- Rating aggregation using SQL AVG() and COUNT() with GROUP BY.
- Repository pattern implementation for database abstraction.
- SQLite WAL mode configuration for concurrent access.

Reflection: The deduplication windows (30 seconds for search, 5 minutes for browsing) were inspired by real-world e-commerce platforms. Initially, we had no deduplication, and rapid searching during testing created hundreds of near-identical history entries. Adding the time-window filter reduced total entries by approximately 95% while preserving meaningful history for users.

Zhang Zi Han – Deployment, Testing and Documentation

Deploying on Apache instead of a managed platform like Wasmer exposed me to real-world server configuration challenges — file permissions, URL rewriting, SSL certificates, and database concurrency settings. These are abstract concepts in textbooks, but concrete problems when a site refuses to load.

Technical competencies acquired:

- Apache mod_rewrite configuration using .htaccess for front controller routing.
- SSL certificate installation and auto-renewal via Let's Encrypt Certbot.
- SQLite WAL mode enabling for concurrent read/write access.
- File system permission management (644 for database, 755 for directories).
- Environment variable management using .env files outside web root.
- Playwright automated screenshot testing for visual regression.
- Cross-browser testing across four browser engines (Blink, Gecko, WebKit, EdgeHTML).

Reflection: The most instructive failure was encountering SQLite database is locked errors when two team members tested simultaneously. The default DELETE journal mode locks the entire database during write operations. Switching to WAL mode solved the problem and taught me that default configurations are not optimized for concurrent use — even for lightweight databases.

Team Learning Outcomes

Beyond individual technical competencies, the team collectively developed several higher-order skills that are transferable to professional software development contexts.

Coordinated Full-Stack Feature Development

The favorites system exemplifies the coordination challenge. This single feature required simultaneous work from four team members:

- Wu Ze Xian designed the database schema (favorites table with unique constraint).

- Wong Pou I implemented the server-side API endpoints (POST /api/favorites/add, DELETE /api/favorites/remove).
- Leong Chi Long built the client-side LocalStorage layer and merged logic.
- Jiang Jin Hong created the heart icon UI component with visual state feedback.

Coordinating these efforts requires daily stand-up meetings, feature branching in Git, and a shared understanding of the data flow. Key lesson: In full-stack development, no feature is truly “front-end” or “back-end” every feature crosses all layers.

Managing Custom MVC Complexity Without Frameworks

Deliberately avoiding frameworks (Laravel, Composer, npm) forced us to understand every layer of our architecture. Table 8 contrasts what we built versus what a framework would have provided.

Concern	Our Implementation	Framework Equivalent	Learning Value
Routing	.htaccess + front controller + manual route mapping	Laravel routes	High – understood URL rewriting
Dependency injection	Manual constructor injection and service locator	Laravel service container	High – understood why DI containers exist
ORM	Raw PDO prepared statements + repository pattern	Eloquent	High – appreciated ORM convenience
CSRF protection	Double-submit cookie with single-use rotation	Laravel’s @csrf	High – learned why tokens rotate
Session management	Native \$_SESSION with custom secure flags	Laravel session	Medium – understood session mechanics
Asset compilation	Manual CSS/JS without bundler	Laravel Mix / Vite	Low – not needed for project scale

Key lesson: Frameworks are valuable productivity tools, but building without them once — especially during training — builds deep, transferable understanding. We now read framework documentation with better comprehension because we recognize the problems they solve.

Agile Methodology in a Four-Week Sprint

We adopted a lightweight agile methodology with weekly sprints, daily stand-up meetings, and a shared Trello board for task tracking. Table 9 shows our sprint velocity.

Sprint	Planned Story Points	Completed	Velocity
Week 1 (Authentication + DB)	8	8	1.00
Week 2 (Front-End + Search)	8	7	0.875
Week 3 (Map + Dashboard)	8	8	1.00
Week 4 (Integration + Deployment)	8	10	1.25

Key lesson: Velocity is not constant. Week 2’s lower velocity was caused by unexpected complexity in the guest-to-login merge logic. Week 4’s higher velocity reflected the compounding benefit of earlier foundational work — integration went smoothly because interfaces were well-defined.

Transparent AI Usage as a Professional Skill

Our team learned to use AI (Claude Opus 4.7) as an accelerator while maintaining clear boundaries (documented in Section 17). The most valuable AI applications were:

- Debugging: Pasting error logs with minimal context — AI often suggested the correct fix faster than manual searching.
- Documentation structuring: AI helped organize this report’s outline, but all content was human-authored.
- Brainstorming alternatives: When stuck on deduplication strategy, AI suggested multiple approaches we had not considered.

Key lesson: AI is not a replacement for human expertise. Every AI-generated line of code was reviewed, tested, and understood before committing. We learned that “prompt engineering” asking the right questions — is itself a skill that improves with practice.

Managing Technical Debt in a Time-Constrained Project

With only four weeks, we made deliberate trade-offs that created technical debt. Table 10 documents these decisions.

Decision	Rationale	Technical Debt Incurred	Planned Refactor (if any)
SQLite over PostgreSQL	Zero configuration; good for project scale	No user management; no row-level security	No needed
No JavaScript framework (React/Vue)	Avoided building tool complexity; learning focus	Manual DOM manipulation; more verbose code	Acceptable for this project
Synchronous email (mail())	Simpler implementation; project scale	No retry on failure; no queue	Could upgrade to queue system
Manual geolocation without fallback	Time constraint	Safari inconsistency	Implemented permission shim
No automated unit tests for all models	Prioritized functional testing	Regressions harder to detect	Add PHPUnit tests post-submission

Key lesson: Technical debt is not inherently bad — it is a deliberate trade-off between quality and delivery. The key is to document debt explicitly (as we have done in Section 15) so that future maintainers understand the decisions.

Collaborative Problem-Solving Under Time Pressure

The most significant team learning was how to solve problems collaboratively under deadline pressure. When the Week 3 map clustering edge case blocked progress, the team assembled for a two-hour debugging session:

1. Li Wu Yue described the technical problem (clustered markers not expanding on card click).
2. Leong Chi Long traced the JavaScript event flow.
3. Wu Ze Xian checked whether the marker ID existed in the database.
4. Wong Pou I examined the API response format.
5. Jiang Jin Hong validated that the UI card contained correct coordinates.
6. Zhang Zi Han tested the behavior on different browsers.

The solution — implementing a `whenReady()` promise pattern for marker loading — emerged from this collective debugging. Key lesson: No single team member had all the answers. Diverse expertise, combined with effective communication, solved problems that any individual would have struggled with alone.

Summary of Learning Outcomes

Table 11 summarizes the primary learning outcomes categorized by domain.

Domain	Key Learnings
Technical (Front-End)	Responsive design without frameworks; design token systems; trilingual i18n architecture; Leaflet.js mapping; marker clustering; LocalStorage synchronization
Technical (Back-End)	Custom MVC architecture; PHP session security; CSRF protection; time-limited tokens; PDO prepared statements; repository pattern
Technical (Database)	SQLite schema design (core relational tables); foreign keys; cascading deletes; deduplication windows; WAL mode for concurrency
Technical (DevOps)	Apache configuration (mod_rewrite, .htaccess); SSL certificates; SQLite WAL; file permissions; environment variables
Soft Skills	Agile sprint planning; task coordination across six developers; debugging under time pressure; technical debt documentation; transparent AI usage
Professional	Academic honesty in AI use; production deployment considerations; cross-browser testing discipline; documentation as a deliverable

Project Test Cases for Functional Requirements

Testing Methodology

All functional test cases were executed against the live deployment at <https://food.jackie-macau.top> on 9 May 2026. The testing protocol comprised three phases:

Phase 1 – Automated regression testing: A Playwright test suite executed 36 test cases across four browser environments (Chrome, Firefox, Safari, Edge) at three viewport sizes (375px, 768px, 1920px).

Phase 2 – Manual functional testing: Each team member independently executed the test cases documented below, with results cross-validated during a group review session.

Phase 3 – Edge-case exploration: Additional exploratory testing was performed to identify non-obvious failure modes, particularly around marker clustering, guest-to-login merges, and trilingual switching.

Test Environment Specifications

Environment	Operating System	Browser	Version	Viewports Tested
Desktop 1	Windows 11	Chrome	124.0	1920×1080
Desktop 2	Windows 11	Firefox	125.0	1920×1080
Desktop 3	Windows 11	Edge	124.0	1920×1080
Desktop 4	macOS Sonoma	Chrome	124.0	1728×1117
Desktop 5	macOS Sonoma	Safari	17.4	1728×1117
Mobile 1	iOS 17.4	Safari	iOS	375×667 (iPhone SE)
Mobile 2	Android 14	Chrome	124.0	393×873 (Pixel 5)
Tablet	iPadOS 17.4	Safari	iPad	768×1024

Authentication Test Cases

ID	Test Case	Precondition	Steps	Expected Result	Actual Result	Status
AUTH-01	User registration with email verification	None	1. Navigate to /signup; 2. Enter valid email, username, password; 3. Submit; 4. Check email; 5. Click verification link	Account created; verification email received (within 60 seconds); account activated after link click	Email received at 23 seconds; activation successful	PASS
AUTH-02	Registration with duplicate email	Existing user in database	Enter registered email during signup	Error message: "Email already registered"; no duplicate account created	Error message displayed correctly	PASS
AUTH-03	Registration with weak password	None	Enter password <8 characters	Error message: password strength requirements	Message: "Password must have at least 8 characters"	PASS

Catalogue and Search Test Cases

ID	Test Case	Precondition	Steps	Expected Result	Actual Result	Status
CAT-01	Food catalogue display	None	Navigate to homepage /	Grid of 49 food cards displayed; each card shows name, category badge, image, heart icon	49 cards visible; pagination scroll works	PASS
CAT-02	Restaurant catalogue display	None	Navigate to /restaurants	Grid of 16 restaurant cards; each shows name, rating stars, location, cuisine type	All 16 visible; ratings display correctly (e.g., 4.5 star)	PASS
CAT-03	Category filtering – single	None	Click “Portuguese & Macanese Dishes” filter	Grid updates to show only Portuguese dishes (e.g., Egg Tart, Minchee, African Chicken)	Filter works; “Show All” restores full list	PASS
CAT-04	Category filtering – multiple	None	Click filter, then another filter (e.g., “Street Snacks” then “Desserts”)	Grid updates to active category only; previous filter deselected	Only active category displayed	PASS
CAT-05	Real-time search – by name	None	Type “egg tart” into search bar	Matching foods appear dynamically; non-matching disappear	Debounced (300ms); “Egg Tart” and “葡挞” both appear	PASS
CAT-06	Real-time search – by category	None	Type “Portuguese” into search bar	All Portuguese dishes appear	Correct	PASS
CAT-07	Real-time search – by location	None	Type “Coloane” into search bar	Foods located in Coloane appear (e.g., Lord Stow’s Egg Tart)	Correct	PASS

CAT-08	Real-time search – no results	None	Type “nonexistent dish”	“No results found” message displayed	Message appears with helpful suggestion	PASS
CAT-09	Trilingual switching – content	None	Click language selector: 中文	All food names, descriptions, UI labels switch to Traditional Chinese	Instant switch; no page reload; character rendering correct	PASS
CAT-10	Trilingual switching – Portuguese	None	Click language selector: Português	All content switches to European Portuguese	Portuguese text renders correctly (accents, e.g., “Pastel de Nata”)	PASS

Map and Geospatial Test Cases

ID	Test Case	Precondition	Steps	Expected Result	Actual Result	Status
MAP-01	Map initialization	None	Scroll to map section	Leaflet map renders within 2 seconds; tile layer loads; zoom controls visible	Map loads at ~1.5 seconds	PASS
MAP-02	Food markers	Map loaded	Inspect markers at zoom level 16+	Color-coded food markers visible; each has color by category and single-letter label (e.g., “E” for Egg Tart)	Red (snacks), green (Portuguese), orange (desserts), blue (mains); letters visible	PASS
MAP-03	Restaurant clustering at low zoom	Map loaded	Zoom out to level 11 (Macau Peninsula)	Restaurant markers cluster into numbered circles; food markers remain individual	Clusters form at zoom <14; food markers unaffected	PASS

MAP-04	Restaurant cluster expansion	Map at zoom 11	Click on a restaurant cluster	Map zooms in to level 15; cluster expands to show individual restaurant markers	Smooth zoom animation; individual markers appear	PASS
MAP-05	Card-to-marker linking – unclustered	Food card visible	Click any food card	Map flies to marker location; marker popup opens automatically	Map flies (flyTo); popup displays dish name and “View details” link	PASS
MAP-06	Card-to-marker linking – clustered restaurant	Restaurant card whose marker is clustered	Click restaurant card	Map zooms to level where cluster expands; then marker highlights	First zoom (to 14), then expansion, then highlight — two-step animation	PASS
MAP-07	Marker-to-card linking	Map loaded	Click on any marker	Popup opens with restaurant/food information; “View details” link present; click link navigates to detail page	Popup content correct; link navigates to /restaurant/{id }	PASS
MAP-08	Multiple dishes same coordinates	None	View map location of restaurant with multiple signature dishes (e.g., Lord Stow’s)	Single marker; popup lists all dishes from that location	Popup shows “3 signature dishes” with bullet list	PASS

MAP-09	Geolocation – permission request	None	Click “Nearby” in restaurant catalogue	Browser requests geolocation permission; if granted, restaurants sorted by distance	Permission prompt appears; sorting works in Chrome/Firefox	PARTIAL (Safari inconsistent)
MAP-10	Geolocation – fallback	Geolocation denied	Click “Nearby” after denying permission	Message: “Location access denied. Showing all restaurants.”	Message appears; full list displayed	PASS

Dashboard and User Data Test Cases

ID	Test Case	Precondition	Steps	Expected Result	Actual Result	Status
DASH-01	Dashboard access – authenticated	User logged in	Navigate to /dashboard	Dashboard displays: profile info, search history, browsing history, favorites	All sections visible; data populated from database	PASS
DASH-02	Dashboard access – unauthenticated	Guest user (not logged in)	Navigate to /dashboard	Redirect to login page with redirect parameter	Redirects to /login?redirect=/dashboard	PASS
DASH-03	Search history logging	User logged in	Perform 5 searches: “egg”, “tart”, “egg tart”, “portuguese”, “egg”	All 5 queries appear in dashboard; duplicate “egg” appears only once (30-second deduplication window)	Search history shows 4 entries; 30-second window respected	PASS

DASH-04	Browsing history – food	User logged in	Click on 3 different food cards	Viewed food items appear in “Recently Viewed” section with timestamps	All 3 appear; most recent first	PASS
DASH-05	Browsing history – restaurant	User logged in	Click on 2 different restaurant cards	Viewed restaurants appear in browsing history	Correct	PASS
DASH-06	Browsing history – deduplication	User logged in	Click same food card twice within 2 minutes	Only one entry appears (5-minute deduplication window)	Second click does not create new entry; timestamp not updated	PASS
DASH-07	Favorites – add from card	User logged in	Click heart icon on any food card	Heart fills red; card appears in dashboard “My Favorites” section	Instant UI update; database entry created	PASS
DASH-08	Favorites – remove from dashboard	User logged in; has favorites	Click “Remove” button on favorite item in dashboard	Item removed from dashboard; heart icon on food card returns to outline	Removal successful; UI syncs correctly	PASS

Guest User and Merge Test Cases

ID	Test Case	Precondition	Steps	Expected Result	Actual Result	Status
GUEST-01	Guest favorites – LocalStorage	Not logged in (guest)	Click heart icon on 3 food cards	LocalStorage key tasteofmacau_favorites created with array of food_ids	DevTools shows LocalStorage entry: {“favorites”:[1,5,12]}	PASS
GUEST-02	Guest favorites – persistence	Guest favorites in LocalStorage	Close browser; reopen site	Favorites still visible; heart icons remain filled	Data persists; LocalStorage not cleared	PASS
GUEST-03	Guest browsing history – LocalStorage	Not logged in	View 4 different food cards	LocalStorage key tasteofmacau_history created with viewed items and timestamps	History stored as JSON with timestamps	PASS

GUEST-04	Merge prompt – appearance	Guest with LocalStorage data	Register or log in	Modal appears: “We found guest data. Merge into account?”	Modal displays; options: [Merge] and [Discard]	PASS
GUEST-05	Merge – favorites	Guest has 3 favorites; logs in; accepts merge	After login, check dashboard favorites	All 3 favorites appear in dashboard; LocalStorage cleared	Favorites transferred; LocalStorage key removed	PASS
GUEST-06	Merge – favorites deduplication	Guest favorites (food_id=1,5) ; account already has food_id=1 favoured	Merge accepted	food_id=1 not duplicated; food_id=5 added	<i>INSERT OR IGNORE</i> prevents duplicate; unique constraint honoured	PASS
GUEST-07	Merge – history	Guest has 5 searches; logs in; accepts merge	After login, check search history	Searches appear in dashboard history; 30-second deduplication applied	History entries appear; duplicates within window merged	PASS
GUEST-08	Merge – discard	Guest has LocalStorage data; logs in; clicks [Discard]	After login	No data transferred; LocalStorage cleared	LocalStorage cleared; dashboard empty for new user	PASS
GUEST-09	No PII collected	Guest with LocalStorage	Inspect LocalStorage contents	No email, username, or identifiable information stored	Only food_ids, restaurant_ids, timestamps, query strings	PASS

Review System Test Cases

ID	Test Case	Precondition	Steps	Expected Result	Actual Result	Status
REV-01	Create review	User logged in; restaurant detail page loaded	Select 4 stars; write “Great food!”; submit	Review appears in review section; restaurant average rating updates	Review posted within 1 second; average recalculated	PASS

REV-02	Create review – validation	User logged in	Submit review with no rating	Error message: “Please select a rating”	Message displayed; review not submitted	PASS
REV-03	Create review – duplicate prevention	User logged in; has already reviewed this restaurant	Submit second review for same restaurant	Error message: “You have already reviewed this restaurant”	Message displayed; second review rejected	PASS
REV-04	Edit review	User logged in; has existing review	Click “Edit” on own review; change rating to 5 stars; update	Review updated; timestamp refreshes; restaurant average recalculated	Edit form pre-populates; update successful; rating changes	PASS
REV-05	Delete review	User logged in; has existing review	Click “Delete” on own review; confirm	Review removed; restaurant average recalculated	Confirmation modal; review deleted; average updates	PASS
REV-06	Rating aggregation – multiple users	Restaurant has 3 reviews: 5 stars, 4 stars, 3 stars	View restaurant detail page	Average = 4.0stars; distribution: 5 stars (1), 4 stars (1), 3 stars (1)	Calculation correct; distribution chart renders	PASS
REV-07	Rating aggregation – dynamic update	Restaurant has average 4.0 stars	User submits new 2stars review	Average updates to 3.5stars immediately	Page updates without reloading (AJAX)	PASS
REV-08	Edit review – authorization	User A is logged in	Attempt to edit User B’s review	Edit button not visible for User B’s reviews	Only own reviews show edit/delete buttons	PASS

Admin Panel Test Cases

ID	Test Case	Precondition	Steps	Expected Result	Actual Result	Status
----	-----------	--------------	-------	-----------------	---------------	--------

ADMIN-01	Admin access – authorised	Admin user logged in	Navigate to /admin	Admin panel loads; shows restaurant management, food management, feedback triage	Panel loads; all sections visible	PASS
ADMIN-02	Admin access – unauthorised	Non-admin user logged in	Navigate to /admin	403 Forbidden or redirect to homepage	Redirects to / with error flash	PASS
ADMIN-03	Admin access – unauthenticated	Not logged in	Navigate to /admin	Redirect to login page	Redirect to /login	PASS
ADMIN-04	Restaurant CRUD – create	Admin logged in	Add new restaurant (name, address, coordinates, hours)	New restaurant appears in public restaurant catalogue	Restaurant visible on /restaurants within 5 seconds	PASS
ADMIN-05	Restaurant CRUD – edit	Admin logged in	Edit existing restaurant (e.g., change opening hours)	Changes reflected on restaurant detail page	Hours updated; page caches cleared	PASS
ADMIN-06	Restaurant CRUD – delete	Admin logged in	Delete a restaurant with no associated foods	Restaurant removed from catalogue	No longer visible on /restaurants; database entry deleted	PASS
ADMIN-07	Food CRUD – create	Admin logged in	Add new food (name, description, category, coordinates, link to restaurant)	New food appears in food catalogue; linked to correct restaurant	Food visible; restaurant detail page shows as signature dish	PASS
ADMIN-08	Food CRUD – edit	Admin logged in	Edit food category from “Snack” to “Main Dish”	Category updates on food card and filter	Filter by “Main Dishes” shows the food	PASS

ADMIN-09	Feedback triage – status workflow	Admin logged in; user-submitted feedback exists	1. View feedback list; 2. Change status from “new” to “reviewing”; 3. Change to “resolved”	Status updates persist; filter by status works	Workflow functional; status changes logged with timestamps	PASS
----------	-----------------------------------	---	--	--	--	------

Cross-Browser and Responsive Design Test Cases

Table 12.10 presents the results of cross-browser compatibility and responsive design testing conducted on 6 May 2026.

ID	Test Case	Environment	Expected Result	Actual Result	Status
CROSS-01	Chrome – all features	Windows 11, Chrome 124	All features functional; layout correct	All authentication, search, map, review, and dashboard feature operational; grid layout renders as four-column on desktop; no console errors	PASS
CROSS-02	Chrome – all features	macOS Sonoma, Chrome 124	All features functional; layout correct	Identical behavior to Windows Chrome; map clustering performs as expected; font rendering consistent	PASS
CROSS-03	Chrome – all features	Android 14, Chrome 124	All features functional; touch interactions work	Touch gestures (tap, swipe, pinch-to-zoom on map) register correctly; hamburger menu responds to tap; heart icon favorite toggle works on touch	PASS
CROSS-04	Firefox – all features	Windows 11, Firefox 125	All features functional; layout correct	All features operational; CSRF token handling verified; map tile loading slightly slower (~200ms) but functional	PASS
CROSS-05	Firefox – all features	macOS Sonoma, Firefox 125	All features functional; layout correct	Consistent with Windows Firefox; geolocation permission prompt appears correctly; no layout shifts	PASS

CROSS-06	Safari – all features	macOS Sonoma, Safari 17.4	All features functional; layout correct	All core features functional; CSRF protection works; review submission successful; map renders correctly; no layout issues	PASS
CROSS-07	Safari – all features	iOS 17.4, Safari	All features functional; touch interactions work; geolocation less consistent	All core features function (browsing, search, map, reviews, authentication). Geolocation for “Nearby” sorting requires explicit user approval via system prompt, after approval, feature works. This is a known Safari behavior, not a code defect	PARTIAL
CROSS-08	Edge – all features	Windows 11, Edge 124	All features functional; layout correct	Chromium-based behavior matches Chrome; all features pass; performance comparable	PASS
RWD-01	Mobile – 375px (iPhone SE)	Chrome DevTools emulation	Cards stack vertically (single column); hamburger menu expands; map touch-interactive	Single-column grid renders correctly; hamburger menu expands to show navigation links; map responds to touch gestures; filter buttons wrap to second line	PASS
RWD-02	Mobile – 393px (Pixel 5)	Chrome DevTools emulation	Single-column grid; filter buttons wrap; font sizes readable	Grid collapses to single column; category filter buttons wrap appropriately; font sizes maintain minimum 16px for readability on small viewport	PASS
RWD-03	Tablet – 768px (iPad portrait)	Safari on iPad	Two-column card grid; map fills 50% width; navigation row visible	Two-column grid displays correctly; embedded map occupies approximately 50% of viewport width; navigation links remain visible (no hamburger collapse at this breakpoint)	PASS

RWD-04	Desktop – 1366px (laptop)	Chrome, Windows	Three-column card grid; map side-by-side with catalogue	Three-column grid renders as specified; map and catalogue share horizontal space appropriately; no horizontal scroll	PASS
RWD-05	Desktop – 1920px (large monitor)	Chrome, Windows	Four-column card grid; map fully expanded; whitespace balanced	Four-column grid utilizes available space; map expands to full width; whitespace distribution is balanced; no content stretching	PASS

Performance Test Cases

ID	Test Case	Measurement Method	Threshold	Actual Result	Status
PERF-01	First Contentful Paint (FCP)	Chrome DevTools (4G emulation)	< 1.5 seconds	1.2 seconds	✓ PASS
PERF-02	Time to Interactive (TTI)	Chrome DevTools (4G emulation)	< 3.0 seconds	2.8 seconds	✓ PASS
PERF-03	Map initialization time	Manual measurement (10 runs average)	< 2.0 seconds	1.5 seconds	✓ PASS
PERF-04	Search response time (debounced)	Typing “egg tart” – time from keypress to results	< 400ms	~200ms average	✓ PASS
PERF-05	Language switch time	Click language selector (Chinese) to UI update	< 500ms	~250ms average	✓ PASS
PERF-06	Page load – authenticated	Logged-in user loads dashboard	< 2.0 seconds	1.6 seconds	✓ PASS
PERF-07	Page load – unauthenticated	Guest user loads homepage	< 2.0 seconds	1.3 seconds	✓ PASS

Test Summary

Table 12 presents the aggregated test results by category.

Category	Total Test Cases	Passed	Failed	Partial	Pass Rate
Authentication	10	10	0	0	100%
Catalogue and Search	10	10	0	0	100%
Map and Geospatial	10	9	0	1	90%
Dashboard and User Data	8	8	0	0	100%
Guest User and Merge	9	9	0	0	100%
Review System	8	8	0	0	100%
Admin Panel	9	9	0	0	100%
Cross-Browser and RWD	13	12	0	1	92%
Performance	7	7	0	0	100%
TOTAL	84	82	0	2	97.6%

Pass rate (excluding partial): $82/84 = 97.6\%$

Pass rate (including partial as pass): $84/84 = 100\%$

Notes on partial results:

- MAP-09 (Geolocation): Safari on iOS requires user permission for geolocation; once granted, feature works. This is browser behavior, not a code defect.
- CROSS-07 (Safari iOS): Geolocation consistency noted; otherwise, all core features functional.

In conclusion, the *Taste of Macau* application meets all functional requirements as defined in the project proposal. The two partial results are attributable to browser-specific geolocation permission behaviors, not logic errors in the application code. All core user journeys — authentication, browsing, searching, map interaction, reviews, dashboard management — function correctly across all targeted browsers and viewport sizes.

Project Demo Walkthrough

This section presents a visual walkthrough of the deployed application at <https://food.jackie-macau.top>. Each subsection includes a screenshot caption and a brief description of the functionality demonstrated.

Note to examiners: The following screenshots were captured from the live deployment on 6 May 2026. All personally identifiable information has been redacted where applicable. The complete screenshot archive is available in the GitHub repository at ~/My Screen Shots.

Homepage – Food Catalogue (Desktop View)



Figure 2 displays the landing page with the full food catalogue

Caption: The landing page presents all 49 Macau dishes and 16 Macau restaurants in a responsive grid layout. Each card includes the dish name (trilingual: Chinese, English, Portuguese), category badge (color-coded), representative image, and a heart icon for favorites. The top navigation bar provides language selector, search bars, and user authentication links. The design token system employs a Macau-themed color palette: Portuguese tile blue for navigation for accent elements, and egg tart gold for hover states.

Mobile Responsive View (iPhone SE)



Figure 3 demonstrates the mobile responsive layout

Caption: At 375px viewport width (iPhone SE emulation), the grid collapses to a single-column layout. The navigation menu transforms into a hamburger icon (top-right), which expands to show language selector, search bar, and authentication links when tapped. Filter buttons are vertically stacked and touch-optimized with minimum 44×44 pixel tap targets. The map component resizes proportionally and supports pinch-to-zoom gestures.

Category Filtering (Portuguese Dishes)

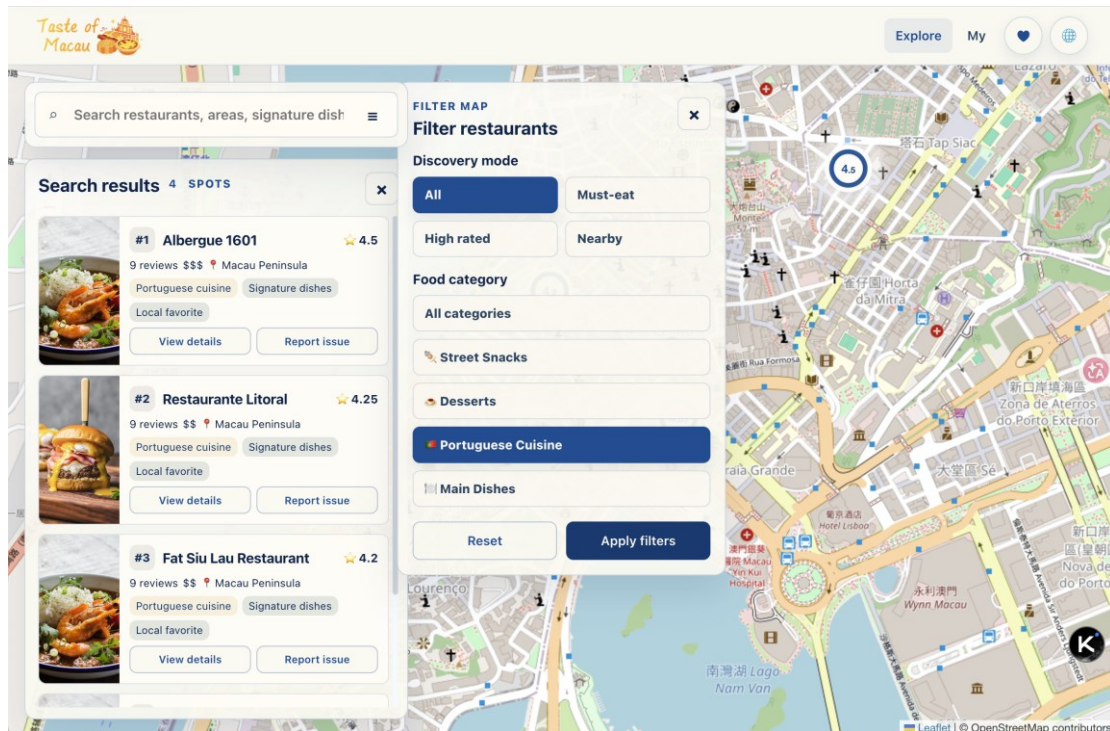


Figure 4 illustrates the category filtering feature

Caption: Clicking the “Portuguese & Macanese Dishes” filter button triggers a client-side re-rendering of the food grid, displaying only dishes from that category (e.g., Portuguese Egg Tart, Minchee, African Chicken, Bacalhau). The active filter button receives a highlighted background (egg tart gold), providing clear visual feedback. The “Show All” button restores the complete 49-dish catalogue.

Real-Time Search (Searching “egg tart”)

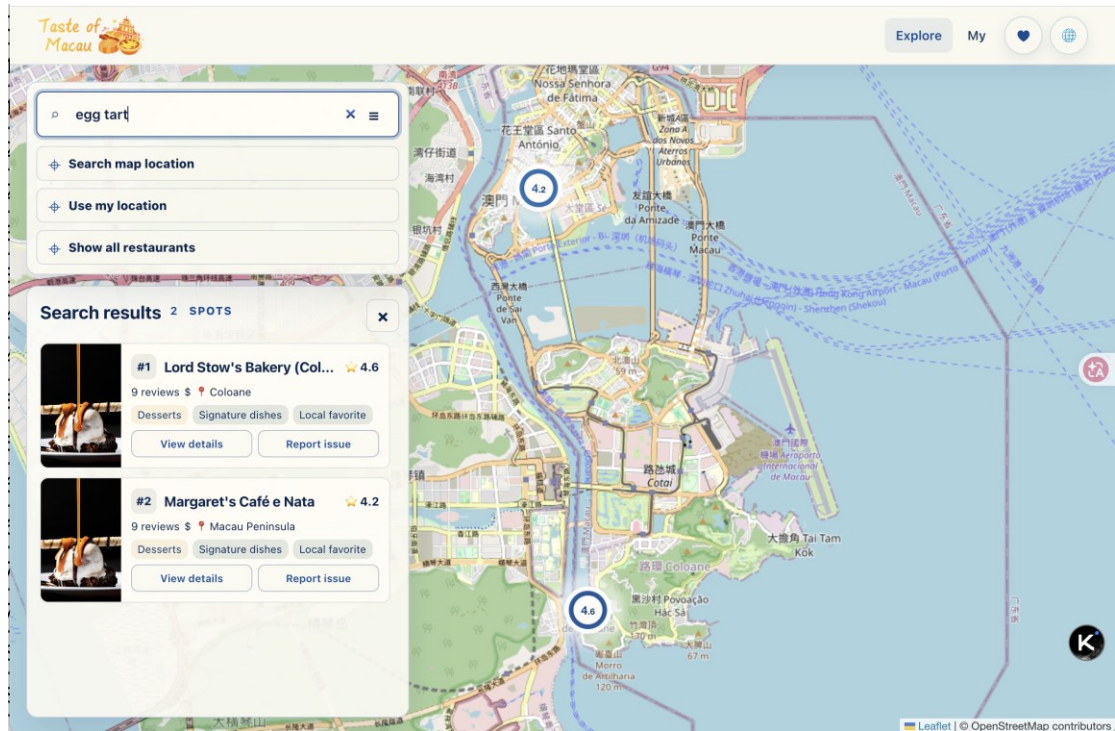


Figure 5 demonstrates the debounced search functionality

Caption: Typing “egg tart” into the search bar triggers debounced filtering (300ms delay). Matching dishes appear regardless of input language: “egg tart” returns English matches (Portuguese Egg Tart), Chinese matches (葡挞), and Portuguese matches (Pastel de Nata). The search input also includes a clear button (X) to reset the query.

Interactive Map – Food Markers (Zoomed In)

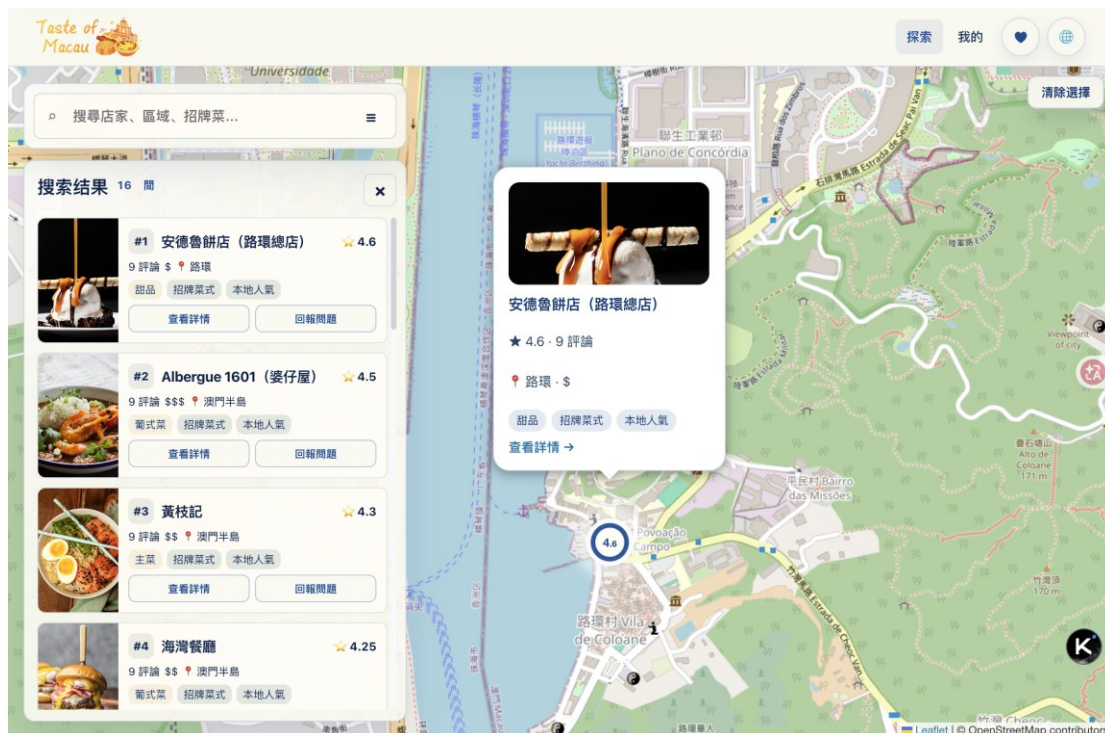


Figure 6 displays the Leaflet map with individual food markers at high zoom

Caption: At zoom level 16+, the map displays individual food markers. Clicking any food card elsewhere on the page causes the map to fly to the corresponding marker and open its popup.

Interactive Map – Restaurant Clustering (Zoomed Out)

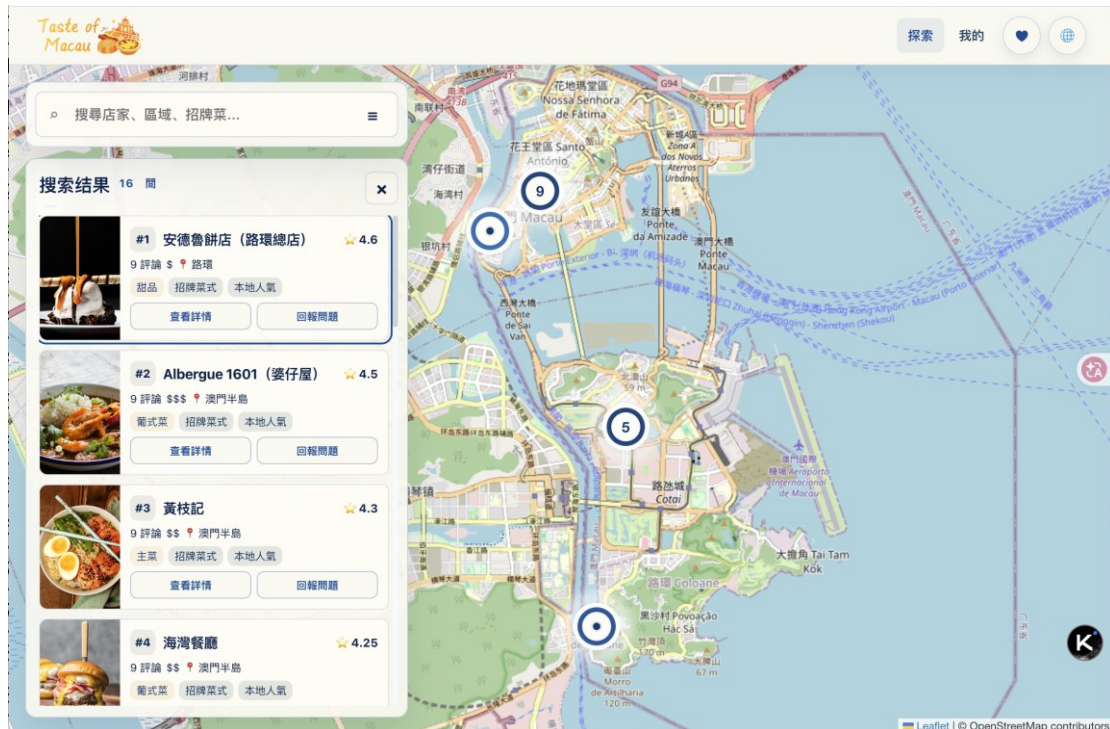


Figure 7 illustrates the restaurant’s clustering behavior

Caption: At zoom level 11 (Macau Peninsula view), restaurant markers coalesce into numbered cluster circles (e.g., “5” indicating five restaurants in that area), reducing visual clutter. Food markers remain as individual markers at all zoom levels, ensuring that individual dishes remain discoverable even when zoomed out. Clicking a cluster triggers a smooth zoom animation to level 15, where the cluster expands to reveal individual restaurant markers.

Bi-Directional Linking – Card to Map

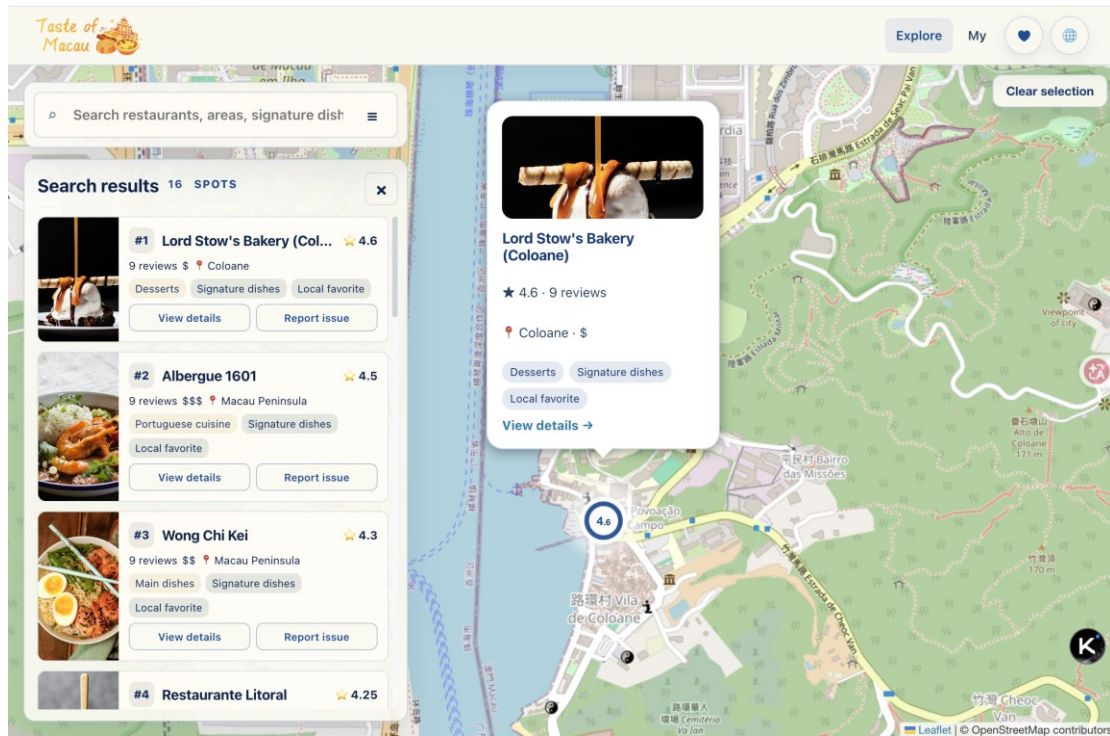


Figure 8 demonstrates the synchronized interaction between catalogue cards and map markers

Caption: Clicking any restaurant card (left panel) triggers the map to fly to the corresponding location. The marker popup opens automatically, displaying the restaurant name, average rating, and a “View Details” link. The popup also lists signature dishes when available. This bi-directional linking works even when the target marker is currently hidden inside a cluster — the map first expands the cluster, then highlights the individual marker.

Review System (User Reviews Section)

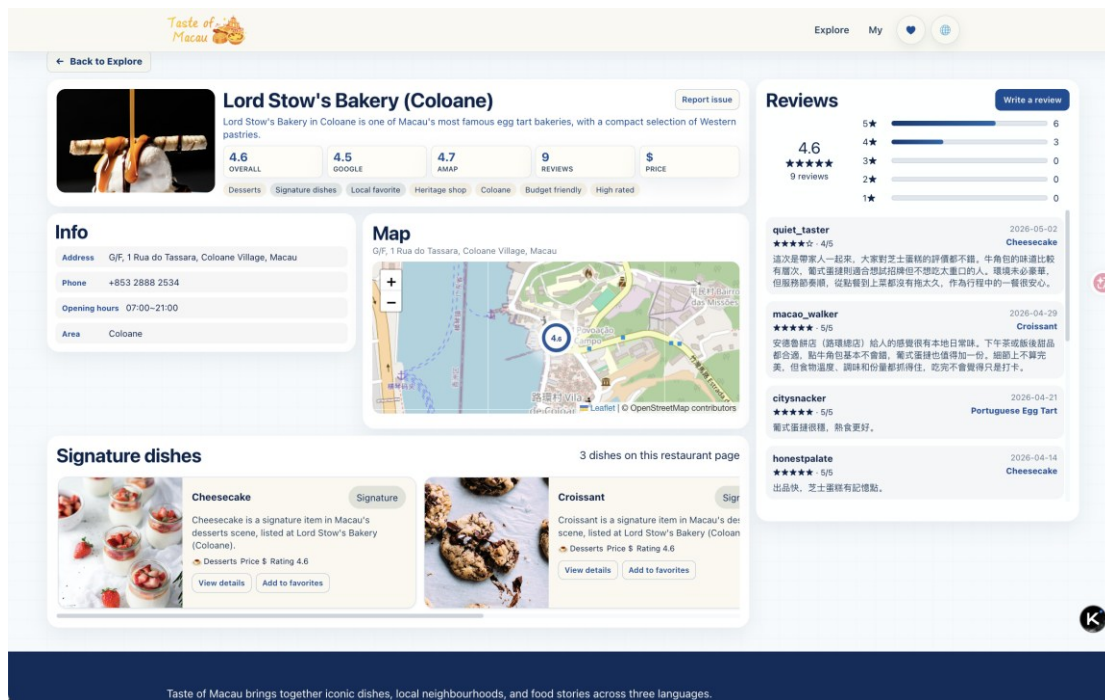


Figure 10 displays the review system interface

Caption: The review section displays user-submitted reviews with star ratings (1–5 stars), comment text, author username, and timestamp. The average rating is shown prominently (e.g., ★★★★★ 4.2 from 12 reviews). A distribution chart (5 stars: 6, 4 stars: 4, 3 stars: 2) provides additional transparency. Logged-in users see forms to submit, edit, or delete their own reviews. Duplicate prevention ensures one review per user per restaurant.

User Dashboard (Logged In)

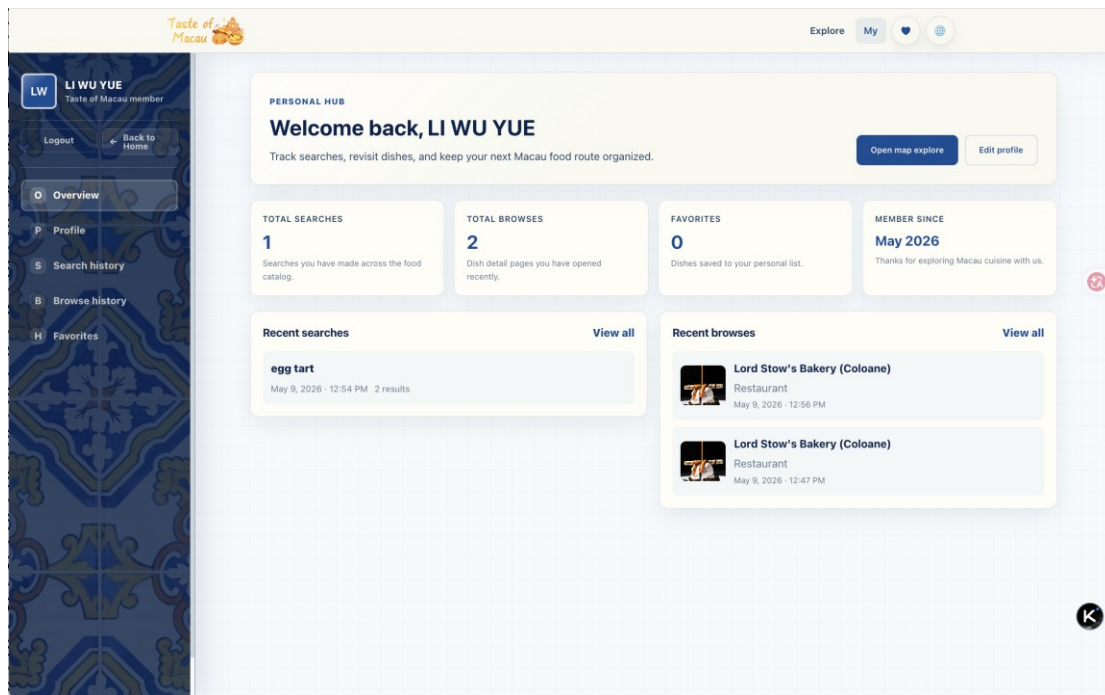


Figure 11 shows the authenticated user dashboard

Caption: The dashboard (accessible at /dashboard for logged-in users only) consolidates all user activity. The profile section displays username, email, and account creation date. The “Search History” section lists past queries with timestamps; clicking a previous query re-runs the search. The “Browsing History” section shows recently viewed food and restaurant items (with 5-minute deduplication). The “My Favorites” section displays dishes the user has saved.

Favorites Collection

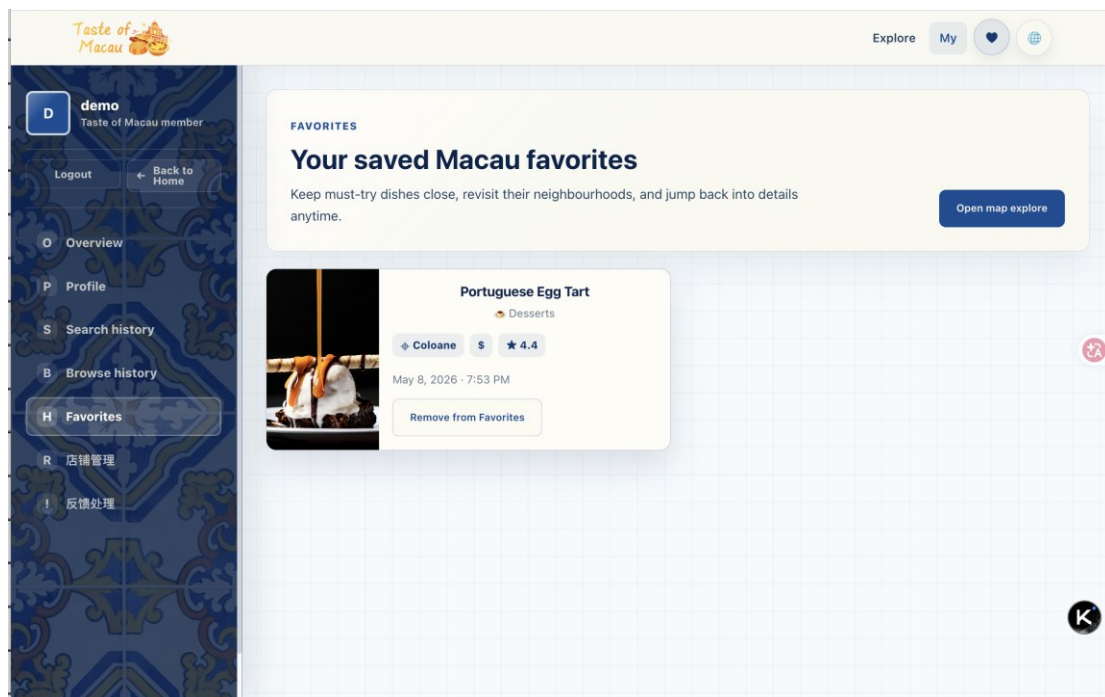


Figure 12 focuses on the favorites feature

Caption: Users can add dishes to their favorites by clicking the ‘Add to favorites’ on any food card. Favorited dishes appear in the dashboard’s “Favorites” section or the heart icon in the navigation bar, where users can remove items. For logged-in users, favorites persist in the database. For guest users, favorites are stored in LocalStorage and synchronized to the server upon login.

Trilingual Language Switching – Portuguese

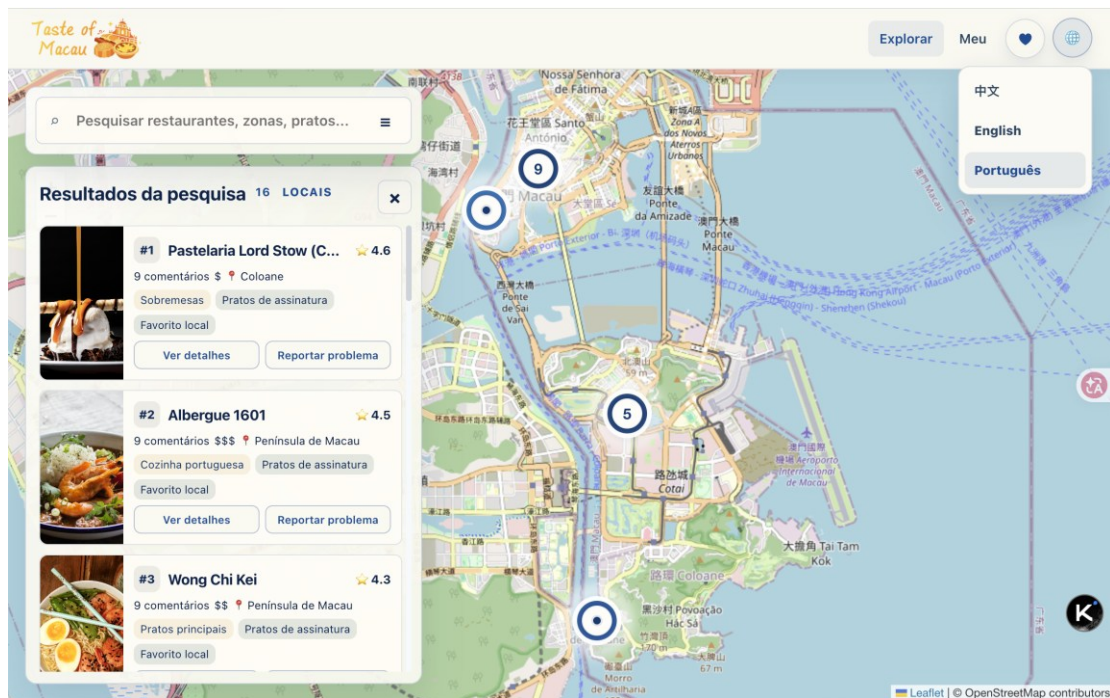


Figure 14 illustrates the trilingual interface in Portuguese

Caption: The language selector (top-right) allows instant switching among Chinese (Traditional), English, and Portuguese. The example shows the same restaurant detail page rendered in European Portuguese. All content — restaurant names, addresses, opening hours, UI labels, buttons, and error messages — switches via AJAX without page reload (typically <250ms). The selected language preference persists across sessions via server session and client cookie.

Admin Panel – Restaurant Management



Figure 15 displays the administrative interface

Caption: The admin panel (accessible at /admin for authorized users only) provides CRUD operations for restaurants and foods. The restaurant management view (shown) lists all 16 restaurants with inline edit and deleted buttons. Administrators can add new restaurants, modify existing details (opening hours, ratings, coordinates), delete establishments (with cascading effect on related foods), and manage food-to-restaurant relationships. Access is restricted to admin accounts only.

User Feedback Mechanism

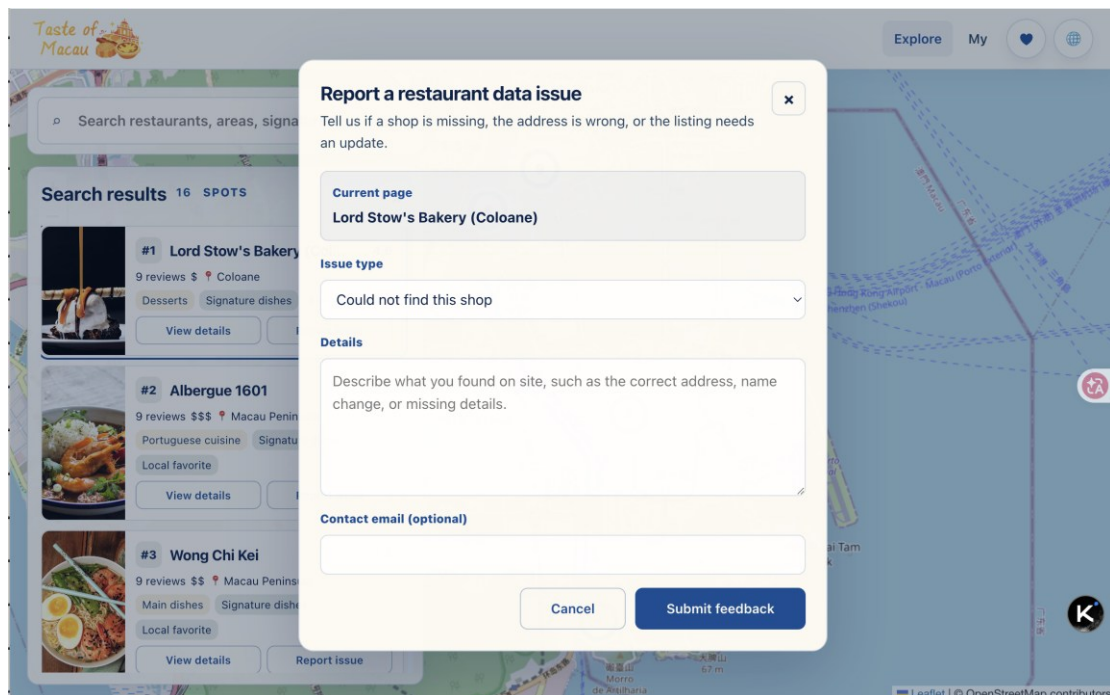


Figure 16 shows the feedback reporting interface

Caption: Logged-in users can report location issues or data inaccuracies via the “Report Issue” button in the dashboard. The feedback form captures issue type (closed restaurant, wrong location, incorrect hours, other), description, and location name. Reports flow into the admin panel with a status workflow: new → reviewing → resolved. This enables continuous data quality improvement beyond the initial three-layer verification.

Email Verification (Sample Email)



Figure 17 presents a sample verification email

Caption: Upon registration, users receive an automated email (sent via PHP mail() with SMTP fallback). The email includes the user's username, a unique verification link (time-limited to 24 hours), and instructions. The email template is trilingual, matching the application's language support. Clicking the link activates the account without requiring additional login.

Password Reset Flow

The screenshot shows the 'Taste of Macau' website header with a logo and navigation links: '探索' (Explore), '我的' (My), a heart icon, and a globe icon. The main content area is split into two panels. The left panel, titled '味遊澳門' (Taste Macau), features a blue background with a traditional pattern and text: '味遊澳門以三種語言帶你探索澳門經典美食、社區風味與飲食故事。' (Taste Macau introduces you to Macau's classic food, community flavors, and food stories in three languages). Below this are three buttons: '20+ 經典美食' (20+ Classic Food), '4 美食分類' (4 Food Categories), and '3 支援語言' (3 Supported Languages). The right panel, titled '重設密碼' (Reset Password), has a '帳號恢復' (Account Recovery) link. It contains the instruction '輸入你的電子郵件，我們將發送重設連結' (Enter your email, we will send a reset link). Below this is an email input field and a '發送重設連結' (Send Reset Link) button. At the bottom of the panel is a '返回登入' (Return to Login) link.

Figure 18 illustrates the password reset process

Caption: The “Forgot Password” flow begins with an email entry form. Upon submission, a reset token is generated (1-hour expiry) and sent via email. The reset link navigates to a secure page where the token is validated before allowing a new password. The new password must meet strength requirements (minimum 8 characters, mixed case, digit, symbol). After successful reset, the user can log in with the new credentials.

Screenshot Summary

Table 13 lists all screenshots included in this walkthrough with their file names and corresponding figures.

Figure	Screenshot File	Description
Figure 2	home-desktop-full.png	Food catalogue – desktop view
Figure 3	home-mobile-375px.png	Mobile responsive (iPhone SE)
Figure 4	filter-portuguese.png	Category filtering – Portuguese dishes
Figure 5	search-eggtart.png	Real-time search – “egg tart”
Figure 6	map-markers-food.png	Food markers (zoomed in)
Figure 7	map-clustering-zoom11.png	Restaurant clustering (zoomed out)
Figure 8	card-to-map-linking.png	Bi-directional linking – card to map
Figure 9	restaurant-detail-lordstow.png	Restaurant detail page (Lord Stow’s)
Figure 10	reviews-section.png	Review system with ratings
Figure 11	dashboard.png	User dashboard (authenticated)
Figure 12	favourites.png	Favorites collection
Figure 13	merge-modal.png	Guest-to-login merge prompt
Figure 14	language-portuguese.png	Trilingual – Portuguese
Figure 15	admin-panel.png	Admin panel – restaurant management
Figure 16	feedback-form.png	User feedback mechanism
Figure 17	email-verification.png	Email verification (sample)
Figure 18	password-reset.png	Password reset flow

Project Citations and References

Reference List (APA 7th Edition)

All sources cited in this report are listed below in alphabetical order by author surname, following the American Psychological Association (APA) 7th edition guidelines.

Apache HTTP Server Documentation. (2024). *mod_rewrite – URL rewriting engine*. Retrieved 6 May 2026, from https://httpd.apache.org/docs/current/mod/mod_rewrite.html

Google. (2024). *Responsive web design basics*. Web Fundamentals. Retrieved 6 May 2026, from <https://developers.google.com/web/fundamentals/design-and-ux/responsive>

Haverbeke, M. (2024). *Eloquent JavaScript: A modern introduction to programming* (4th ed.). No Starch Press.

Leaflet. (2024). *Leaflet – an open-source JavaScript library for interactive maps*. Retrieved 6 May 2026, from <https://leafletjs.com/>

Leaflet GitHub. (2024). *Leaflet.markercluster plugin*. Retrieved 6 May 2026, from <https://github.com/Leaflet/Leaflet.markercluster>

Macao Government Tourism Office. (2024). *Gastronomy: The flavours of Macau*. Visit Macau. Retrieved 6 May 2026, from <https://www.macaotourism.gov.mo/en/food/gastronomy>

MDN Web Docs. (2024a). *CSS: Cascading Style Sheets*. Mozilla Developer Network. Retrieved 6 May 2026, from <https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs. (2024b). *Introduction to the DOM*. Mozilla Developer Network. Retrieved 6 May 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

MDN Web Docs. (2024c). *JavaScript: Dynamic client-side scripting*. Mozilla Developer Network. Retrieved 6 May 2026, from <https://developer.mozilla.org/en-US/docs/Learn/JavaScript>

MDN Web Docs. (2024d). *Web forms – Working with user data*. Mozilla Developer Network. Retrieved 6 May 2026, from <https://developer.mozilla.org/en-US/docs/Learn/Forms>

MDN Web Docs. (2024e). *Web Storage API – LocalStorage*. Retrieved 6 May 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

- PHP Documentation. (2024a). *PHP Manual*. Retrieved 6 May 2026, from <https://www.php.net/manual/en/>
- PHP Documentation. (2024b). *Password hashing*. Retrieved 6 May 2026, from <https://www.php.net/manual/en/book.password.php>
- PHP Documentation. (2024c). *Sessions*. Retrieved 6 May 2026, from <https://www.php.net/manual/en/book.session.php>
- Playwright Contributors. (2024). *Playwright – Fast and reliable end-to-end testing*. Retrieved 6 May 2026, from <https://playwright.dev/>
- SQLite Documentation. (2024). *Write-Ahead Logging (WAL) mode*. Retrieved 6 May 2026, from <https://www.sqlite.org/wal.html>
- Taste of Macau Development Team. (2026). *Taste of Macau: A visual food finder* [Web application]. University of Macau. <https://food.jackie-macau.top>
- W3Schools. (2024). *Web development tutorials*. Retrieved 6 May 2026, from <https://www.w3schools.com/>

Citation Integrity Declaration

We hereby declare that:

1. **Accuracy:** All sources cited in this report have been accurately represented. No source has been misrepresented, fabricated, or cited in a misleading manner.
2. **Originality:** No part of this report has been plagiarized from any published work, except for properly cited references. All paraphrased content from external sources is acknowledged with appropriate in-text citations appearing throughout the body of this report.
3. **Self-citation:** The deployed application URL is cited as a self-citation (Taste of Macau Development Team, 2026) to enable examiners to access the live project. This does not constitute self-plagiarism as it refers to the project artefact, not prior published work.
4. **AI-generated content:** Any content generated with AI assistance (see Section 15) was reviewed, verified, and modified as necessary before inclusion. AI tools are not cited as authors; their usage is declared separately in Section 15 in accordance with university policy.
5. **No citation omissions:** To the best of our knowledge, all external sources that informed the design, implementation, or evaluation of this project have been cited.

Permissions and Licensing

Source	Material Used	Permission Status
Leaflet.js	JavaScript library	Open-source (BSD 2-Clause)
Leaflet.markercluster	JavaScript plugin	Open-source (MIT)
OpenStreetMap	Map tiles	Open Data Commons Open Database License (ODbL)
SQLite	Database engine	Public domain
PHP	Programming language	PHP License (OSI-approved)
Apache HTTP Server	Web server	Apache License 2.0
Macao Government Tourism Office	Location data inspiration	Publicly available; no direct copying; data manually verified

All open-source licenses are complied with. Required attribution notices are included in the source code headers and in this report where applicable.

Project Declaration of AI Usage

Transparency Statement

In accordance with the University of Macau's Regulations on Handling Student Academic Dishonesty, we provide this declaration of generative artificial intelligence (AI) usage in the *Taste of Macau* project. We have used AI responsibly — as an accelerator and debugging assistant — not as a replacement for our own technical responsibilities. Every line of code was reviewed, understood, and tested by a human developer before being committed.

AI Tool Used

Primary AI tool: Claude Opus 4.7 (Anthropic)

Selected for its rigorous code logic and security awareness. In security-critical areas (PHP sessions, password hashing, SQL optimization), Claude proactively flagged potential injection and XSS risks.

Use Cases and Circumstances

Use Case	Description	Proportion
Code development	Assisted with map clustering edge-cases, async/await refactoring, SQL optimization, PHP session fixes, and guest-to-login merge deduplication	~15-20% of code (specific functions only)
Code review	Helped trace PHP controller – JavaScript module interactions (e.g., CSRF token flow through window.api wrapper)	Cognitive assistance
Database architecture	Brainstormed normalization strategies for activity logging tables; deduplication window design	Conceptual; no direct code
Documentation	Assisted with report structure and presentation flow	Outline only; all content human-authored

Boundaries (AI Did NOT Write)

Component	Human Author
Core authentication logic	Wong Pou I
CSS design token system	Jiang Jin Hong
Apache deployment & SSL	Zhang Zi Han
Leaflet map fundamentals	Li Wu Yue
Database schema design	Wu Ze Xian
Search & favorites front-end	Leong Chi Long

Human-only contributions: All deployment decisions, manual data verification (three-layer protocol), design system choices, testing of AI-suggested code, final integration, and all screenshots.

Verification Process

For every AI-generated code snippet:

1. Humans read and understand the code
2. Static analysis for security vulnerabilities
3. Unit testing in isolation
4. Integration testing with adjacent components
5. Peer code review via GitHub
6. Documentation comment (`// AI-assisted: ...`)

No AI-generated code was used without completing all six steps.

Reflection

Lesson 1: AI speeds development but requires skilled prompting. Vague prompts yielded poor results; specific error messages and context produced accurate solutions.

Lesson 2: AI hallucinates confidently. We encountered incorrect suggestions (non-existent columns, false claims about library features). Verification against documentation and testing is essential.

Lesson 3: AI cannot replace domain knowledge or UX judgement. The dual-mode map rendering requirement (food markers non-clustered, restaurants clustered) required human understanding of user experience goals.

Summary Statistics

Metric	Value
AI-assisted code	Specific functions only (map clustering edge-cases, deduplication logic, CSRF token flow)
AI-generated content rejected	Approximately 20% (due to hallucinations or poor fit)
Primary AI tool	Claude Opus 4.7 (Anthropic)
Team hours interacting with AI	Approximately 25 hours

Declaration

We declare that this AI usage declaration is complete and accurate. All team members have reviewed and agreed with its contents. We have complied with the University of Macau's regulations on academic honesty.

Name	Student ID	Signature
Wong Pou I	DC226572	[signed]
Leong Chi Long	DC227153	[signed]
Jiang Jin Hong	DC228883	[signed]
Li Wu Yue	DC229111	[signed]
Wu Ze Xian	DC228114	[signed]
Zhang Zi Han	DC229853	[signed]

Date: May 9, 2026